



New Mansoura University
Faculty of Computer Science and Engineering

Structured Programming

(CSE014)

Lecture Notes

Dr. Mohamed Handosa

Fall 2024

Contents

1	Introduction	1
1.1	Computer System Components	1
1.2	Computer System Organization	2
1.3	Computer Software	3
1.4	Programming Languages	4
1.5	Structured Programming	4
1.6	Unstructured Programming	5
1.7	Structured versus Unstructured	5
1.8	Algorithms	7
1.9	Problem Solving	8

1.10 Pseudocode	10
1.11 Flowcharts	11
1.11.1 Terminator	11
1.11.2 Process	12
1.11.3 Decision	12
1.11.4 Data Input/O	12
1.11.5 Arrow	13
1.12 Flowchart Examples	13
1.12.1 Flowchart Example 1	13
1.12.2 Flowchart Example 2	14
1.12.3 Flowchart Example 3	15
2 Basic Syntax	17
2.1 Program Structure	17
2.2 The <code>using</code> Directive	21
2.3 Write and WriteLine	22

2.4	Comments	23
2.4.1	Single-Line Comments	23
2.4.2	Multi-Line Comments	24
2.5	Literals	25
2.6	Variables	26
2.6.1	Defining Variables	27
2.6.2	Defining Multiple Variables	30
2.7	Constants	30
2.8	Identifiers	32
3	Data Types	33
3.1	Boolean Variables	34
3.2	Integer Variables	35
3.3	Unsigned Integer Variables	36
3.4	Integer Overflow	36
3.5	Floating-Point Numbers	38

3.6	Character Variables	40
3.7	String Variables	41
3.8	Value-Types and Reference-Types	43
3.8.1	Value-Types	43
3.8.2	Reference-Types	44
3.9	Type Conversion	45
3.9.1	Implicit Type Conversion	45
3.9.2	Explicit Type Conversion	47
3.9.3	Type Conversion Methods	49
3.10	User Input	51
4	Operators	57
4.1	Sign operators	57
4.2	Arithmetic Operators	58
4.3	Increment and Decrement Operators	60
4.4	Assignment Operators	62

4.5	Comparison Operators	63
4.6	Logical Operators	64
4.7	Expressions	65
4.8	Conditional Operator	66
4.9	String Concatenation	67
4.10	Escape Sequences	68
4.11	Operator Precedence	69
4.12	Operator Associativity	70
4.13	Operand Evaluation	72
5	Branching	75
5.1	The <code>if</code> Statement	75
5.2	The <code>if...else</code> Statement	78
5.3	Nested <code>if</code> Statements	82
5.4	The <code>switch</code> Statement	86
5.4.1	The <code>break</code> Statement	88

5.4.2	The <code>default</code> Case	89
5.5	Scope of Variables	90
6	Looping	93
6.1	The <code>while</code> Loop	93
6.2	The <code>do..while</code> Loop	97
6.3	The <code>for</code> Loop	101
6.4	The <code>break</code> Statement	107
6.5	The <code>continue</code> Statement	109
6.6	Nested Loops	111
7	Arrays	119
7.1	Creating Arrays	120
7.2	Accessing Array Elements	121
7.3	Array Literal	122
7.4	Arrays and Loops	123
7.5	The <code>foreach</code> Loop	126

7.6	Copying Arrays	128
7.7	Multidimensional Arrays	134
8	Methods	141
8.1	Creating a Method	142
8.2	Calling a Method	143
8.3	Parameters and Arguments	146
8.4	Multiple Parameters	147
8.5	Return Values	148
8.6	Default Arguments	150
8.7	Named Arguments	152
8.8	Method Overloading	154
8.9	Passing Arguments	156
8.10	Passing Arrays	159
8.10.1	Passing a Reference By Value	159
8.10.2	Passing a Reference By Reference	163

9 Case Study**167**

Chapter 1

Introduction

1.1 Computer System Components

A computer system can be divided into four components: the **hardware**, the **operating system**, the **application programs**, and the **users** as shown in Figure 1.1.

- **Hardware:** provides the basic computing resources for the system (e.g., the Central Processing Unit (CPU), the memory, and the Input/Output (I/O) devices).
- **Application programs:** define the ways in which these resources are used to solve users' computing problems (e.g., word processors, spreadsheets, compilers, and Web browsers).

- **Operating system:** controls the hardware and coordinates its use among the various application programs for the various users (e.g., Microsoft Windows, Apple iOS, Google Android).
- **Users:** commonly humans but can be machines as well.

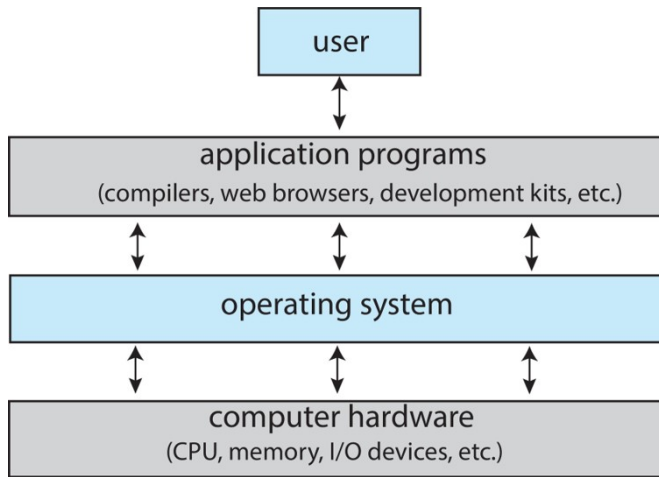


Figure 1.1: Abstract view of the components of a computer system.

1.2 Computer System Organization

A modern general-purpose computer system consists of one or more CPUs (or processors) and a number of I/O devices connected through a common bus that provides access to shared memory, as shown in Figure 1.2. The user can provide input using an input device and receive output via an output device.

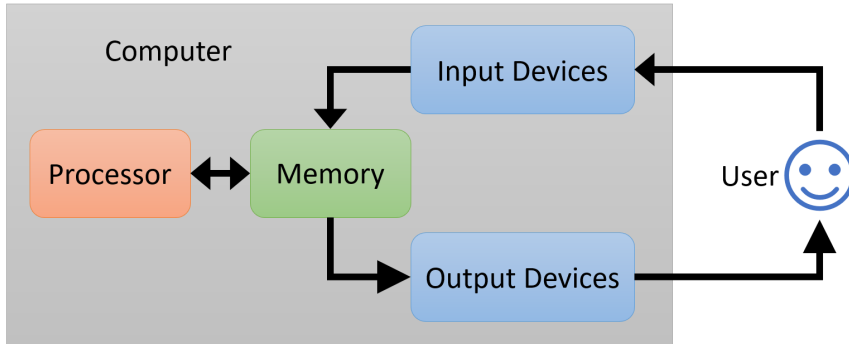


Figure 1.2: A modern computer system.

1.3 Computer Software

Computer software, or simply software, is a collection of data or computer instructions that tell the computer how to work. This is in contrast to physical hardware, from which the system is built and actually performs the work.

In computer science and software engineering, computer software is all information processed by computer systems, programs and data. Computer software includes computer programs, libraries and related non-executable data, such as online documentation or digital media.

Computer hardware and software require each other and neither can be realistically used on its own. Computer hardware is like a car and computer software is like the driver of that car.

A **program** is a collection of instructions to perform a specific task.

1.4 Programming Languages

A **computer program** is developed by a **computer programmer**. The programmer writes the **source code** representing the program instructions using one of the **programming language**.

There are several programming languages with different syntax and features. The programmer must obey the rules of the programming language in use.

Once the source code is ready, it can be converted into a computer program using the **compiler** of the programming language in use, as shown in Figure 1.3. The compiler itself is a program that can convert source code into an executable program.

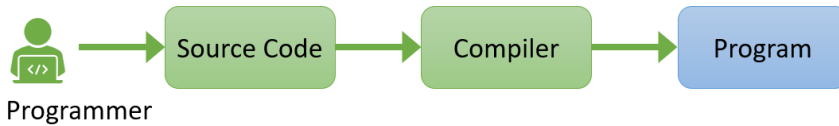


Figure 1.3: The role of a compiler.

1.5 Structured Programming

In Structured Programming, the code is divided into functions or modules. It is also known as modular programming.

Modules or functions are a set of statements which performs a sub task. As each task is a separate module, it is easy for the programmer to test and debug. It is also easy to do modifications without changing the whole program.

When changing the code, the programmer has to concentrate only on the specific module. C# language and Java are some examples of structured programming languages.

1.6 Unstructured Programming

In Unstructured Programming, the code is written as a single whole block. The whole program is taken as a single unit. It is harder to do changes in the program.

This paradigm was used in earlier versions of BASIC, COBOL, and FORTRAN. Unstructured programming languages have a limited number of data types like numbers, arrays, and strings.

1.7 Structured versus Unstructured

Structured and unstructured programming are two paradigms in programming. The difference between Structured and Unstructured pro-

programming is that Structured programming languages allow the programmer to divide the whole program into modules or functions and in Unstructured programming, the program is written as one single block.

Structured programming languages are the modern languages, and unstructured languages are the earliest versions of programming languages. There are differences between structured and unstructured programming in terms of readability, purpose, complexity, application, modification, data types, code duplication, and testing.

- **Readability.** Structured Programming based programs are easy to read. Unstructured Programming based programs are hard to read.
- **Purpose.** Structured Programming is to make the code more efficient and easier to understand. Unstructured programming is just to program to solve the problem. It does not create a logical structure.
- **Complexity.** Structured Programming is easier because of modules. Unstructured programming is harder when comparing with the structured programming.
- **Application.** Structured programming can be used for small and medium scale projects. Unstructured programming is not applicable for medium and complex projects.
- **Modification.** It is easy to do changes in Structured Programming. It is hard to do modifications in Unstructured Program-

ming.

- **Data types.** Structured programming uses many data types. Unstructured programming has a limited number of data types.
- **Code duplication.** Structured programming avoids code duplication. Unstructured programming can have code duplication.
- **Testing and debugging.** It is easy to do testing and debugging in Structured Programming. It is hard to do testing and debugging in Unstructured programming.

1.8 Algorithms

An algorithm is simply a set of steps used to complete a specific task. They're the building blocks for programming, and they allow things like computers, smartphones, and websites to function and make decisions.

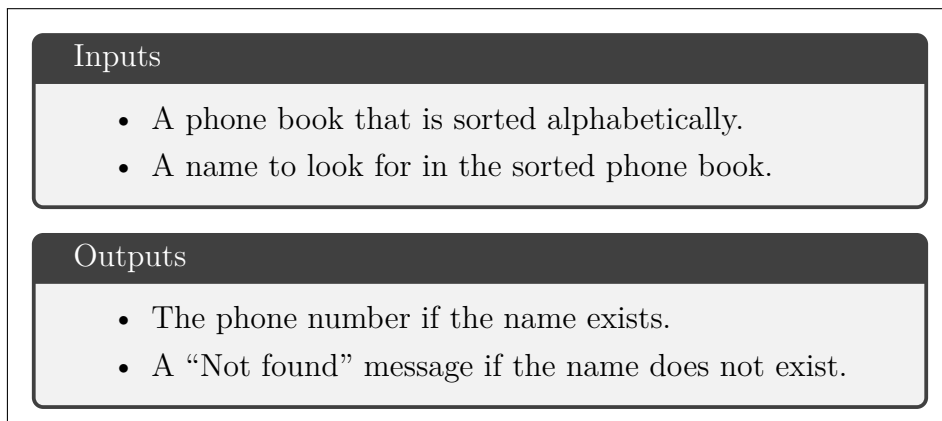
In addition to being used by technology, a lot of things we do on a daily basis are similar to algorithms. Let's say you want to make some boiled eggs. In order to do this successfully, there's a certain set of steps you need to follow in a particular order.

First, you'll need to boil a pot of water. Once it's boiling, you then add the eggs and cook it for a set amount of time. Once it's finished, you drain the water and peel the eggs, then they are ready to be served.

Humans and computers can use algorithms to perform everyday tasks. The difference is that computers can use algorithms and calculate things better, faster, and more efficiently than we can.

1.9 Problem Solving

Solving problems is the core of computer science. Programmers must first understand how a human solves a problem, then understand how to translate this “algorithm” into something a computer can do, and finally how to “write” the specific syntax (required by a computer) to get the job done. Sometimes, a machine will solve a problem in a completely different way than a human. Consider the following example of a problem that needs to be solved:



In order to develop a program that can solve this problem, we need

to think of an algorithm that a human (or a computer) can follow to solve this problem. The following are three candidate algorithms:

Algorithm 1

Start from the first page, looking for the name one page at a time. This algorithm is correct but slow. Searching for a name in a phone book of 1000 pages can take up to 1000 steps (if the name does not exist in the phone book).

Algorithm 2

Start from the first page, flip through the book two pages at a time. This algorithm is faster than **Algorithm 1**. Searching for a name in a phone book of 1000 pages can take up to only 500 steps. However, this algorithm is incorrect as it may skip the page containing the name we are searching for.

Algorithm 3

Open the phone book to the middle and decide which half contains the name. Repeat this and divide the problem in half each time until we find the name. With 1024 pages to start, we will only need 10 steps to find the name if it exists. This algorithm is both correct and fast.

1.10 Pseudocode

Pseudocode is a plain language description of the steps in an algorithm or another system. Pseudocode often uses structural conventions of a normal programming language, but is intended for human reading rather than machine reading.

Consider the following example of a Pseudocode describing an algorithm that takes two numbers as inputs, adds them, and provides their sum as output.

```
1 BEGIN
2 NUMBER s1, s2, sum
3 OUTPUT("Input number 1:")
4 INPUT s1
5 OUTPUT("Input number 2:")
6 INPUT s2
7 sum = s1 + s2
8 OUTPUT sum
9 END
```

As shown in the example above, Pseudocode is an English-language-like representation of an algorithm. It does not have a standard syntax and can take different forms. It is a compact and informal high-level description of a program. It is not a source code that follows the syntax rules of some programming language. Therefore, it cannot be compiled into a program.

1.11 Flowcharts

A flowchart is simply a graphical representation of steps. It shows steps in sequential order and is widely used in presenting the flow of algorithms, workflow or processes. It shows the steps as boxes of various kinds, and their order by connecting them with arrows.

It was originated from computer science as a tool for representing algorithms and programming logic but had extended to use in all other kinds of processes. Nowadays, flowcharts play an extremely important role in displaying information and assisting reasoning. They help us visualize complex processes, or make explicit the structure of problems and tasks.

Different flowchart shapes have different conventional meanings. Those shapes include terminator, process, decision, and data I/O besides arrows that connect them.

1.11.1 Terminator

The terminator symbol represents the starting or ending point of the system.



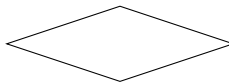
1.11.2 Process

A box indicates some particular operation.



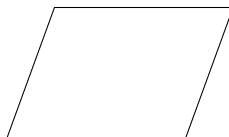
1.11.3 Decision

A diamond represents a decision or branching point. Lines coming out from the diamond indicates different possible situations, leading to different sub-processes.



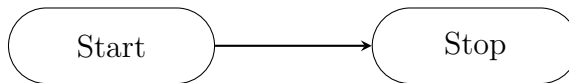
1.11.4 Data Input/O

It represents information entering or leaving the system. An input might be an order from a customer. Output can be a product to be delivered.



1.11.5 Arrow

Lines represent the flow of the sequence and direction of a process.

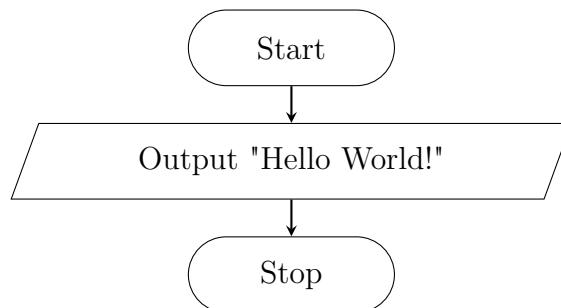


1.12 Flowchart Examples

The following examples show flowcharts that represent simple programs.

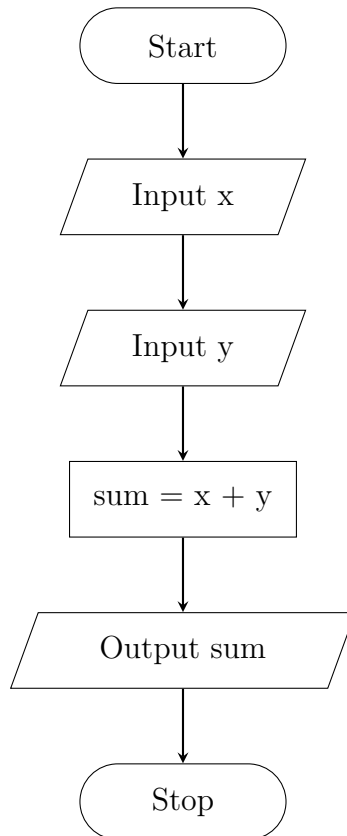
1.12.1 Flowchart Example 1

The following flowchart illustrates the required steps to print the sentence “Hello World” on the screen.



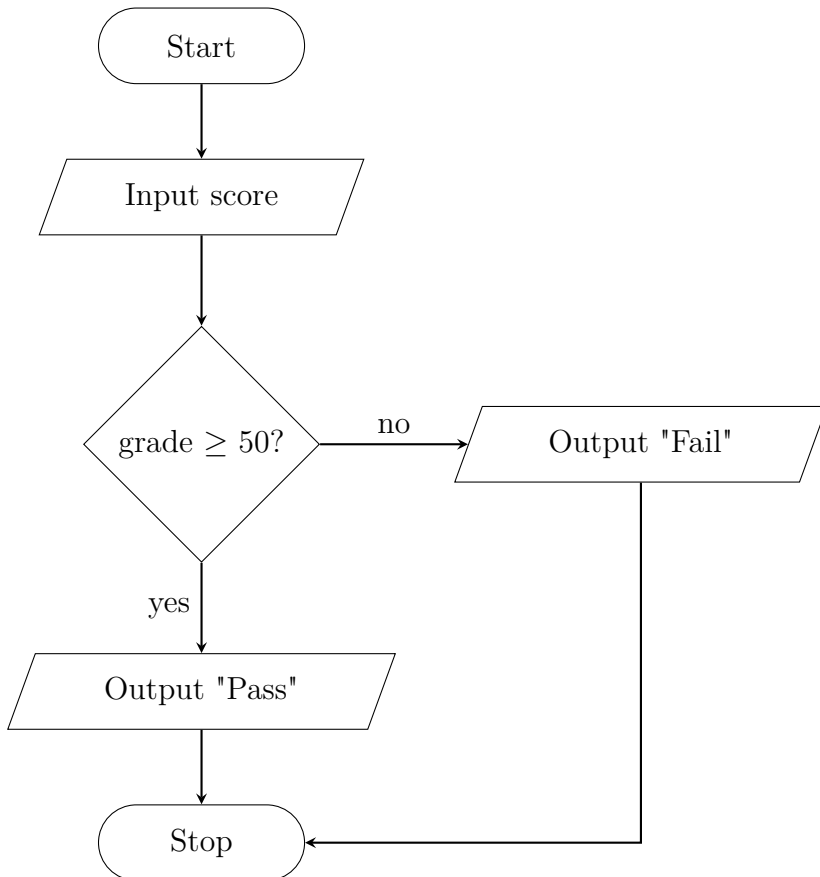
1.12.2 Flowchart Example 2

The following flowchart illustrates the required steps to input two numbers, calculate their sum, and print the result on the screen.



1.12.3 Flowchart Example 3

The following flowchart illustrates the required steps to input the score of a student. Afterward, print “Pass” to the screen if the score is equal to or greater than 50. Otherwise, print “fail” to the screen.



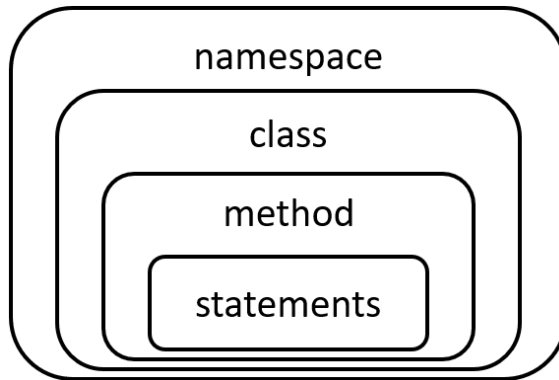
Chapter 2

Basic Syntax

In this lecture, we take the very first steps towards learning structured programming using the C# programming language.

2.1 Program Structure

A C# program consists of a set of one or more namespaces, each **namespace** contains a set of one or more classes. A **namespace** may contain other namespaces as well. Each **class** contains a set of one or more methods. Each **method**, in turn, may contain a set of one or more statements and each statement represents a command to execute. The following figure shows the code structure of a C# program.



The following example shows a simple C# program with a single statement that prints the text `Hello World!` to the screen:

```
1 namespace HelloWorld
2 {
3     class Program
4     {
5         static void Main()
6         {
7             System.Console.WriteLine("Hello World!");
8         }
9     }
10 }
```

Click to run ►

Output

Hello World!

In the above example, line 1 defines a **namespace** named `HelloWorld` followed by two curly braces (an opening curly brace `{` in line 2 and a corresponding closing curly brace `}` in line 10). These two curly braces mark the beginning and the end of the **namespace**. A **namespace** is used to organize your code. It works as a container for **classes** and other **namespaces**. Any code that is written within the two curly braces of a **namespace** is contained within that **namespace** (i.e. the code lines 3-9 in the above example are contained within the `HelloWorld` namespace).

Line 3 in the above code defines a **class** named `Program`. A class is a container for data and methods. Again, the two curly braces (the opening curly brace `{` in line 4 and the corresponding closing curly brace `}` in line 9) define the beginning and the end of the **class**.

Lines 5-8 define a **method** named `Main`. Any code lines within the `Main` method (i.e. within its curly braces) will be executed when the program runs. In our example, we have a single line of code within the `Main` method, which is

```
1 System.Console.WriteLine("Hello World!");
```

This line of code prints the text `Hello World!` to the screen by calling a library **method** named `Write`. Note that `"Hello World!"` is enclosed within double quotation marks. In C#, **string** literals

must be enclosed within double quotation marks. The `Write` library method itself is contained within a library class named `Console`, which in turn is contained within a namespace named `System`. Therefore, when we call the `Write` method, we must specify its fully qualified name by writing `System.Console.Write` followed by two braces `()` containing the data that we want to pass to the `Write` method, which is the string literal `"Hello World!"` in our example.

In C#, every code statement must end with a semicolon `;`. Note that line 7 in our example ends with a semicolon `;`.

C# is case-sensitive. If you change line 7 in the above example by modifying the name of the method from `Write` to `write` as follows

```
1 System.Console.write("Hello World!");
```

The C# compiler will not compile the code and will show the following error message:

Error Message

'Console' does not contain a definition for 'write'

This is because the words `Write` and `write` are considered different words in C#. The C# compiler will provide an error message if there is something wrong with your code.

2.2 The using Directive

The `using` directive allows you to use everything that is defined in a namespace without specifying the fully qualified namespace.

In its basic form, the `using` directive imports everything from a single namespace, as shown in the following example:

```
1  using System;
2
3  namespace HelloWorld
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.WriteLine("Hello World!");
10         }
11     }
12 }
```

Click to run ►

In the above code, line 1 uses the `using` directive to import everything that the `System` namespace contains. Note that in line 9 we no longer need to specify the namespace `System` before using the `Console` class and its `Write` method.

2.3 Write and WriteLine

In C#, both `Write` and `WriteLine` are methods in the `Console` class that can be used to output something to the console.

The only difference between the `Write` and `WriteLine` is that `Console.Write` is used to print data without printing the new line character, while `Console.WriteLine` is used to print data along with printing the new line character afterward. This would set a new line for the next output. For example, consider the following program:

```
1  using System;
2
3  namespace PrintNumbers
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write(1);
10             Console.WriteLine(2);
11             Console.WriteLine(3);
12             Console.Write(4);
13         }
14     }
15 }
```

Click to run ►

In the above code, line 9 uses the `Write` method to print `1` to the screen. The `Write` method will not print a new line character

after printing `1`. Therefore, the next output will be printed to the same line. Line 10 uses the `WriteLine` method to print `2` to the screen. The `WriteLine` method will print a new line character after printing `2`. Therefore, the next output will be printed on a new line. Similarly, line 11 uses the `WriteLine` method to print `3` followed by a new line character. Finally, line 12 uses the `Write` method to print `4` to the screen. The output of this program will be:

Output
12
3
4

2.4 Comments

Comments can be used to explain C# code, and to make it more readable. It can also be used to prevent the execution of a portion of code when testing alternative code.

2.4.1 Single-Line Comments

Single-line comments start with two forward slashes `//`. Any text between `//` and the end of the line is ignored by C#.

An example of a single-line comment before a line of code:

```
1 // This is a comment
2 System.Console.WriteLine("Hi!");
```

An example of a single-line comment at the end of a line of code:

```
1 System.Console.WriteLine("Hi!"); // This is a comment
```

2.4.2 Multi-Line Comments

Multi-line comments start with `/*` and ends with `*/`. Any text between `/*` and `*/` will be ignored by C#.

An example of a multi-line comment to explain the code:

```
1 /* The code below
2 will print the words
3 Hello World
4 to the screen. */
5 System.Console.WriteLine("Hello World!");
```

2.5 Literals

Literals refer to fixed values that the program may not alter during its execution. They can be of any of the basic data types like an integer literal, a floating-point literal, a character literal, or a string literal. For example:

```
1 namespace Literals
2 {
3     class Program
4     {
5         static void Main()
6         {
7             System.Console.WriteLine(5);
8             System.Console.WriteLine(5.2);
9             System.Console.WriteLine(5.2E-3);
10            System.Console.WriteLine('A');
11            System.Console.WriteLine("Hi");
12        }
13    }
14 }
```

Click to run ►

In the above example, line 7 prints the integer literal `5` to the screen. Line 8 prints the floating-point literal `5.2` to the screen. A floating-point literal has an integer part, a decimal point, and a fractional part. It can also have an exponent part as in line 9, where `5.2E-3` means `5.2 × 10-3` or `0.0052`.

Line 10 prints the character `A` to the screen, Note that a character literal must be enclosed in single quotation marks (i.e. `'A'`). Finally, line 11 prints the string `Hi` to the screen. Note that a string literal must be enclosed in double quotation marks (i.e. `"Hi"`).

2.6 Variables

Variables are containers for storing data values. In C#, there are different types of variables (defined with different keywords). The following table shows some keywords that can be used to define variables.

Keyword	Description
<code>int</code>	Defines an integer variable that can store a whole number, without a fraction (e.g. 123 or -123)
<code>double</code>	Defines a variable that can store a floating-point number, with a fraction, (e.g. 19.99 or -19.99).
<code>char</code>	Defines a character variable that can store a single character (e.g. 'a' or 'B').
<code>string</code>	Defines a string variable that can store a text of any length (e.g. "Hello World").
<code>bool</code>	Defines a Boolean variable that can store one of two values; either <code>true</code> or <code>false</code> .

There are many other keywords that can be used to define other types of variables. You will learn about them later in chapter 3.

2.6.1 Defining Variables

To define a variable, you must specify a type and a name. The general syntax takes the following form:

Syntax

type name;

The following example defines an integer variable:

```
1 namespace Variables
2 {
3     class Program
4     {
5         static void Main()
6         {
7             int x;
8             x = 5;
9             System.Console.WriteLine(x);
10        }
11    }
12 }
```

Click to run ►

In the above example, line 7 defines a variable named `x` of type `int`. Line 8 uses the assignment operator `=` to store the value `5` into the variable `x`. Finally, line 9 uses the `WriteLine` method to print the variable `x` to the screen. This will print the value of the variable to the screen (i.e. `5`) and the output of the program will be:

Output

5

You can initialize a variable when defining it. That is you can provide an initial value to the variable. In the previous example, lines 7 and 8 can be merged into a single line of code as follows:

```
1  int x = 5;
```

In the above code, line 1 defines a variable named `x` of type `int` and assigns it an initial value of `5`.

Note that if you assign a new value to an existing variable, it will overwrite the previous value. For example:

```
1  int x = 5;  
2  x = 6;
```

In the above code, line 1 defines a variable named `x` of type `int` and assigns it the value `5`. Later, line 2 assigns the value `6` to the same variable `x`. This means that the old value `5` that was stored in the variable `x` will be lost and the new value of the variable `x` will be `6`.

The following example creates several variables of different types and prints their values to the screen:

```
1  using System;
2
3  namespace Literals
4  {
5      class Program
6      {
7          static void Main()
8          {
9              int count = 5;
10             double price = 5.2;
11             char grade = 'A';
12             string name = "Mohamed Handosa";
13             bool pass = true;
14             Console.WriteLine(count);
15             Console.WriteLine(price);
16             Console.WriteLine(grade);
17             Console.WriteLine(name);
18             Console.WriteLine(pass);
19         }
20     }
21 }
```

Click to run ►

In the above code, line 9 defines a variable named `count` of type `int` and assigns it a value of `5`. Line 10 defines a variable named `price` of type `double` and assigns it a value of `5.2`. Line 11 defines a variable named `grade` of type `char` and assigns it a value of `'A'`. Line 12 defines a variable named `name` of type

`string` and assigns it a value of `"Mohamed Handosa"`. Line 13 defines a variable named `pass` of type `bool` and assigns it a value of `true`. Finally, lines 14-18 print the values of those variables to the screen using the `WriteLine` method.

2.6.2 Defining Multiple Variables

To define more than one variable of the same type, use a comma-separated list as in the following example:

```
1 int x, y, z;
```

In the above code, line 1 defines three variables of type `int`. The variables are named `x`, `y`, and `z`. You can give some of your variables initial values when defining them as in the following example:

```
1 int x = 5, y, z = 50;
```

2.7 Constants

You can use the `const` keyword if you don't want others (or yourself) to overwrite the existing value of a given variable.

The general syntax to define a constant is:

Syntax

```
const type name = value;
```

For example, the following code will result in an error:

```
1  const int x = 15;  
2  x = 20; // error
```

In the above code, line 1 defines a constant named `x` of type `int` and assigns it the value `15`. Later, line 2 tries to modify the value of the constant `x` to `20`. However, the C# compiler will produce an error message because the value of a constant cannot be altered.

The `const` keyword is useful when you want a variable to always store the same value. An example that is often referred to as a constant, is `PI` (i.e. $22 \div 7$ or 3.14159).

Note that you cannot declare a constant variable without assigning it a value. If you do, the C# compiler will produce the following error message:

Error Message

```
A const field requires a value to be provided.
```

2.8 Identifiers

All C# variables must be identified with unique names. These unique names are called identifiers. Identifiers can be short names (e.g., `x` and `y`) or more descriptive names (e.g., `age`, `totalPrice`).

It is recommended to use descriptive names in order to create understandable and maintainable code. For example:

```
1  int minutesPerHour = 60; // A good variable name
2  int m = 60; // Not so easy to guess what m actually is
```

The general rules for creating variable names (unique identifiers) are:

- A name cannot contain a white-space.
- A name must begin with a letter or an underscore `_`.
- A name can contain only letters, digits and the underscore.
- Keywords (e.g. `int`, `double`, `string`) cannot be used as names.
- Names are case-sensitive (e.g. `size` and `Size` are different).

The following are invalid variable names (i.e. identifiers) that will result in compile-time errors if used:

```
1  string 1stName; // Cannot start with a digit.
2  int total count; // Cannot contain a white space.
3  double price@store; // Cannot contain a special character.
```

Chapter 3

Data Types

A data type specifies the size of a variable as well as the type of the data it can store. It is important to use the correct data type for your variables. This can help avoid errors and save memory. Moreover, it makes the code more maintainable and readable.

Computer memory consists of a list of bytes. Each byte consists of 8 bits, where each bit can hold either a 0 or a 1. A variable consists of one byte, two bytes, four bytes, or eight bytes. The data type of the variable specifies the number of bytes constituting it.

C# provide several data types allowing programmers to create variables that can store Boolean values, integer values, floating-point values, characters, and strings.

3.1 Boolean Variables

A Boolean variable can hold one of two values: `true` or `false`. The size of a Boolean variable is `1` byte. A Boolean variable can be defined using the keyword `bool` or using `System.Boolean` as shown in the table below.

Data Type	.NET Name	Size
<code>bool</code>	<code>System.Boolean</code>	1 byte

The following example creates two Boolean variables and print their values to the screen:

```
1  using System;
2
3  namespace BooleanVariables
4  {
5      class Program
6      {
7          static void Main()
8          {
9              bool x = true;
10             Boolean y = false;
11             Console.WriteLine(x);
12             Console.WriteLine(y);
13         }
14     }
15 }
```

Click to run ►

In the above code, line 9 defines a Boolean variable named `x` using the `bool` keyword while line 10 defines another Boolean variable named `y` using the .NET name `Boolean`.

3.2 Integer Variables

An integer value is a whole number without a fraction. It can be positive, zero, or negative. C# provides several data types of different sizes that can be used to define integer variables. These data types are shown in the following table:

Data Type	.NET Name	Size	Min Value	Max value
sbyte	System.SByte	1 byte	-2^{07}	$+2^{07} - 1$
short	System.Int16	2 bytes	-2^{15}	$+2^{15} - 1$
int	System.Int32	4 bytes	-2^{31}	$+2^{31} - 1$
long	System.Int64	8 bytes	-2^{63}	$+2^{63} - 1$

As shown in the above table, the smallest available size to store an integer value is the signed byte `sbyte`, which consists of 1 byte (8-bits). This data type uses a single bit to store the sign (0 for positive and 1 for negative) and uses the remaining 7 bits to store the value. The largest available size to store an integer value is `long`, which consists of 8 bytes (64-bits). This data type uses a single bit to store the sign and uses the remaining 63 bits to store the value.

3.3 Unsigned Integer Variables

C# provides data types that can only hold non-negative integer values. These data types are shown in the following table:

Data Type	.NET Name	Size	Min Value	Max value
byte	System.Byte	1 byte	0	$+2^{08} - 1$
ushort	System.UInt16	2 bytes	0	$+2^{16} - 1$
uint	System.UInt32	4 bytes	0	$+2^{32} - 1$
ulong	System.UInt64	8 bytes	0	$+2^{64} - 1$

As shown in the above table, a `byte` variable can hold a larger integer number compared to a signed byte `sbyte` variable. This is because a `byte` variable does not have to store a sign and uses all of its 8-bits to store the value. Similarly, `ushort` uses all of its 16-bits to store a larger value compared to `short`. The same goes for `uint` and `int` as well as `ulong` and `long`. Remember that these data type cannot be used to create variables to store negative numbers because these variables will not have a sign bit.

3.4 Integer Overflow

An integer overflow occurs when an arithmetic operation attempts to create a numeric value that is outside of the range that can be repre-

sented with a given number of bits (either higher than the maximum or lower than the minimum value that can be represented). For example, executing the following code will result in an integer overflow:

```

1 namespace Overflow
2 {
3     class Program
4     {
5         static void Main()
6         {
7             uint x = 4294967295;
8             uint y = x + 5;
9             System.Console.WriteLine(y);
10        }
11    }
12 }
```

Click to run ►

In the above code, line 7 assigns the value 4294967295 to the `uint` variable named `x`. This value is actually the maximum value that a `uint` variable can hold ($2^{32} - 1$). Line 8 calculates the sum of `x` and `5`, which should be $2^{32} + 4$. However, this summation value is too large to fit within the 32 bits of a `uint` value. Therefore, an integer overflow occurs and the bit representing the value 2^{32} is lost. This leaves the `y` variable with a value of `4`, which is not correct.

	11111111	11111111	11111111	11111111	$2^{32} - 1$
+	00000000	00000000	00000000	00000101	5
<code>1</code>	00000000	00000000	00000000	00000100	4

3.5 Floating-Point Numbers

An floating-point value consists a whole number with a fraction. It can be positive, zero, or negative. C# provides several data types of different sizes that can be used to define floating-point variables. These data types are shown in the following table:

Data Type	.NET Name	Size	Precision
float	System.Single	4 bytes	6-9 digits
double	System.Double	8 bytes	15-17 digits
decimal	System.Decimal	16 bytes	28-29 digits

Decimal values have much higher precision and are usually used in financial applications that require a high degree of accuracy. However, processing decimal values is much slower compared to processing float and double values.

The type of a floating-point literal is determined by its suffix as follows:

- The literal without suffix is of type `double`
- The literal with the `f` or `F` suffix is of type `float`
- The literal with the `d` or `D` suffix is of type `double`
- The literal with the `m` or `M` suffix is of type `decimal`

The following example demonstrates the use suffixes and shows the precision of each floating-point data type:


```
1 using System;
2
3 namespace RealNumbers
4 {
5     class Program
6     {
7         static void Main()
8         {
9             float x = 1F / 3F;
10            Console.WriteLine(x);
11
12            double y = 1D / 3D;
13            Console.WriteLine(y);
14
15            decimal z = 1M / 3M;
16            Console.WriteLine(z);
17        }
18    }
19 }
```

Click to run ▶

Executing the above code will result in the following output:

Output

0.33333334

0.3333333333333333

0.33333333333333333333333333333333

3.6 Character Variables

A character variable can hold a single character. The size of a character variable is `2` bytes. A character variable can be defined using the `char` or using `System.Char` as shown in the table below.

Data Type	.NET Name	Size
<code>char</code>	<code>System.Char</code>	2 bytes

UTF-16 is a character encoding standard, where each character is represented by a unique 16-bit sequence of 0s and 1s. In C#, the `char` type represents a UTF-16 character. For example,

```
1 namespace Characters
2 {
3     class Program
4     {
5         static void Main()
6         {
7             char ch = 'A';
8             ushort num = ch;
9             System.Console.WriteLine(ch);
10            System.Console.WriteLine(num);
11        }
12    }
13 }
```

Click to run ►

In the above code, line 7 defines a variable named `ch` of type `char` and assigns it the value `'A'`. Line 8 copies the contents of the `ch` variable into the `num` variable. Note that `ch` and `num` have the same size that is 2 bytes. Line 9 prints the contents of the `ch` variable to the screen. It prints `A` because `ch` is of type `char`. Line 10 prints the contents of the `num` variable to the screen. It prints `65` because `num` is of type `ushort`.

Note that both `ch` and `num` store the same exact sequence of 0s and 1s but the data type of each variable decides the way the content of that variable is interpreted and printed to the screen. The output of executing the above program will be:

Output
A
65

3.7 String Variables

A `string` variable is a **reference-type** variable. It can hold a **reference** to a sequence of characters stored in memory. The size of a `string` variable depends on the computer system. On a 32-bit computer system, the size of a `string` variable is 4 bytes. On a 64-bit computer system, the size of a `string` variable is 8 bytes.

A string variable can be defined using the `string` keyword or using `System.String` as shown in the following table.

Data Type	.NET Name	Size
string	System.String	4 bytes or 8 bytes

The following example defines a string variable and outputs its value:

```
1  namespace Text                                     Click to run ►
2  {
3      class Program
4      {
5          static void Main()
6          {
7              string text = "Hi";
8              System.Console.WriteLine(text);
9          }
10     }
11 }
```

Output

Hi

3.8 Value-Types and Reference-Types

There are two kinds of types in C#: value types and reference types. A **value-type** variable directly contain the data while a **reference-type** variable stores a reference to the data.

3.8.1 Value-Types

C# has several value-types including Boolean type (i.e. `bool`), character type (i.e. `char`), integral numeric types (i.e. `sbyte`, `short`, `int`, and `long`), unsigned integral numeric types (i.e. `byte`, `ushort`, `uint`, and `ulong`), and floating-point numeric types (i.e. `float`, `double`, and `decimal`).

A **value-type** variable stores a data value in its memory space. For example, suppose that we define a variable of type `int` named `x` and assign the value `5` to it as follows:

```
1 int x = 5;
```

In the above code, line 1 allocates a memory space of 4 bytes to represent a variable of type `int` named `x`. Afterward, it stores the value `5` in those 4 bytes. Now, the memory space of the variable named `x` directly contains the data (i.e. the value `5`).

3.8.2 Reference-Types

C# has several reference-types such as the `string` type. There are other reference-types such as **arrays** and **objects**.

A **reference-type** variable stores a reference in its memory space. This reference is the address of the memory location where the data is stored. For example, suppose that we define a variable of type `string` named `s` and assign the value `"Hello"` to it as follows:

```
1 string s = "Hello";
```

In the above code, line 1 allocates a memory space of 4 (or 8) bytes to represent a variable of type `string` named `s`. Afterward, it allocates another memory space to store the data and stores the value `"Hello"` into it. Finally, it stores the address of the memory space containing the data into the variable named `s`. Now, the memory space of the variable named `s` contains a reference to the data (i.e. the address of the memory location storing the value `"Hello"`).

Computer Memory	Address:8400	Address:1480
	1480	"Hello"
	Variable <code>s</code>	Actual data

3.9 Type Conversion

Type conversion refers to changing a value from one data type to another. In C#, type conversion can be implicit or explicit.

3.9.1 Implicit Type Conversion

An implicit type conversion can be made when the value can fit into the new data type without losing any data. For example, a value of type `int` (32-bit integer) can safely fit into a variable of type `long` (64-bit integer). No special syntax is required because the conversion always succeeds and no data will be lost. For example,

```
1 namespace ImplicitConversion
2 {
3     class Program
4     {
5         static void Main()
6         {
7             int x = 50;
8             long y = x; // Implicit Conversion
9             System.Console.WriteLine(x);
10            System.Console.WriteLine(y);
11        }
12    }
13 }
```

Click to run ►

In the above example, line 8 is trying to assign the value stored in the `int` variable named `x` to the `long` variable named `y`. In other words, line 8 is trying to make a copy of the value stored in `x` (i.e. 50) and store that copy in `y`. Note that the variable `x` is of type `int`. Therefore, when we make a copy of the value stored in `x` that copy will be of type `int` as well.

In order to store the `int` copy in the `long` variable, the compiler implicitly converts the `int` copy to a `long` value before assigning it to `y`. Note that the variable `x` itself and the value stored in it are still of type `int`. The compiler will just convert the copy of `x` from `int` to `long` before assigning it to `y`. The output of the above program will be:

Output

```
50
50
```

A given value can be implicitly converted to a data type if it can represent that value without losing any data. For example:

- `char` → `ushort`
- `byte` → `short`, `ushort` → `int`, `uint` → `long`
- `sbyte` → `short` → `int` → `long` → `float` → `double`
- `byte` → `ushort` → `uint` → `ulong` → `float` → `double`

3.9.2 Explicit Type Conversion

If a conversion cannot be made without a risk of losing information, the compiler requires that you perform an explicit conversion, which is called a cast. A **cast** is a way of explicitly informing the compiler that you intend to make the conversion and that you are aware that data loss might occur. For example, a value of type `double` cannot fit into a variable of type `int` because the fraction will be lost. Therefore, such conversion requires a cast.

To perform a cast, specify the type that you are casting to in parentheses in front of the value or variable to be converted. The following program casts a `double` to an `int`. The program will not compile without the cast.

```
1 namespace ExplicitConversion
2 {
3     class Program
4     {
5         static void Main()
6         {
7             double x = 5.8;
8             int y = (int)x; // Casting
9             System.Console.WriteLine(x);
10            System.Console.WriteLine(y);
11        }
12    }
13 }
```

Click to run ►

In the above example, line 8 is trying to assign the value stored in the `double` variable named `x` to the `int` variable named `y`. In other words, line 8 is trying to make a copy of the value stored in `x` (i.e. 5.8) and store that copy in `y`. Note that the variable `x` is of type `double`. Therefore, when we make a copy of the value stored in `x` that copy will be of type `double` as well.

In order to store the `double` copy in the `int` variable, the compiler explicitly converts the `double` copy to an `int` value before assigning it to `y`. Note that the variable `x` itself and the value stored in it are still of type `double`. The compiler will just convert the copy of `x` from `double` to `int` before assigning it to `y`. The output of the above program will be:

Output

5.8

5

The casting expression in line 8 (i.e. `(int)`) has explicitly forced the compiler to perform the conversion. Note that this explicit conversion has resulted in losing data (i.e. the fraction) because the `double` value did not fit in the `int` variable. Sometimes, a programmer might want this behaviour (e.g. to remove the fraction part from a `double` value).

3.9.3 Type Conversion Methods

The C# library provide a set of methods that can be used to convert a value from one data type to another.

Method	Description
<code>System.Convert.ToSByte</code>	Returns a value of type <code>sbyte</code>
<code>System.Convert.ToInt16</code>	Returns a value of type <code>short</code>
<code>System.Convert.ToInt32</code>	Returns a value of type <code>int</code>
<code>System.Convert.ToInt64</code>	Returns a value of type <code>long</code>
<code>System.Convert.ToByte</code>	Returns a value of type <code>byte</code>
<code>System.Convert.ToUInt16</code>	Returns a value of type <code>ushort</code>
<code>System.Convert.ToUInt32</code>	Returns a value of type <code>uint</code>
<code>System.Convert.ToUInt64</code>	Returns a value of type <code>ulong</code>
<code>System.Convert.ToSingle</code>	Returns a value of type <code>float</code>
<code>System.Convert.ToDouble</code>	Returns a value of type <code>double</code>
<code>System.Convert.ToDecimal</code>	Returns a value of type <code>decimal</code>
<code>System.Convert.ToBoolean</code>	Returns a value of type <code>bool</code>
<code>System.Convert.ToChar</code>	Returns a value of type <code>char</code>
<code>System.Convert.ToString</code>	Returns a value of type <code>string</code>

Note that converting a floating-point numeric value (e.g., `float`, `double`, or `decimal`) to an `int` value, for example, using the `System.Convert.ToInt32` method will round the value instead of just removing the fraction. For example,

```
1  using System;
2
3  namespace ConversionMethods
4  {
5      class Program
6      {
7          static void Main()
8          {
9              double x = 5.8;
10             int y = (int)x; // Removes fraction
11             int z = Convert.ToInt32(x); // Rounds fraction
12             Console.WriteLine(x);
13             Console.WriteLine(y);
14             Console.WriteLine(z);
15         }
16     }
17 }
```

Click to run ►

In the above example, line 10 performs a casting, which will remove the fraction altogether. Line 11 uses the `Convert.ToInt32` method, which will round the fraction. The output of the program will be:

Output

5.8

5

6

3.10 User Input

A C# program can accept user input using the `Console.ReadLine` library method, which can be found under the `System` namespace. This method returns user input as a `string` value, which we can assign to a `string` variable as in the following example:

```
1  using System;
2
3  namespace UserInput
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter your name: ");
10             string name = Console.ReadLine(); // User input
11             Console.Write("Your name is ");
12             Console.WriteLine(name);
13         }
14     }
15 }
```

Click to run ►

In the above example, line 9 prints a message asking the user to provide input. Line 10 accepts user input using the `ReadLine` method. Once the user writes the input and presses the `Enter` key, the `ReadLine` method will return that input as a value of type `string`, which is then assigned to the `name` variable. Line 11 prints a message to the

screen before line 12 prints the value of the `name` variable. Assuming that the user input is `Mohamed Handosa`, the output will be:

Output

```
Enter your name: Mohamed Handosa
Your name is Mohamed Handosa
```

Sometimes, you would want a program to accept an `int` value as input. However, the `ReadLine` method returns input as a `string` value. In this case, you can use the `Convert.ToInt32` method to convert the `string` value returned by the `ReadLine` method to a value of type `int`. The following program accepts two integer values from the user and prints their sum to the screen:

```
1  using System;
2
3  namespace Sum
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter 1st number: ");
10             string s1 = Console.ReadLine();
11             int x = Convert.ToInt32(s1);
12
13             Console.Write("Enter 2nd number: ");
14             string s2 = Console.ReadLine();
15             int y = Convert.ToInt32(s2);
16
```

Click to run ►

```
17         int sum = x + y;
18
19         Console.Write("Sum = ");
20         Console.WriteLine(sum);
21     }
22 }
23 }
```

In the above example, line 10 accepts the first user input and stores it as a `string` in a variable named `s1`. Line 11 uses the `ToInt32` method to convert the value of `s1` into an `int` value before storing it in a variable named `x`. Similarly, line 14 stores the second user input as a `string` in a variable named `s2`. Line 15 converts the `string` value stored in `s2` into an `int` value using the `ToInt32` method before storing it in a variable named `y`. Line 17 defines a variable of type `int` named `sum`. Afterward, it uses the `+` operator to add the integer values of `x` and `y` before assigning the result to the variable `sum`. Line 19 and 20 print the output to the screen, which is the sum of the two values entered by the user. Assuming that the user enters `5` and `6`, the output will be:

Output

```
Enter 1st number: 5
Enter 2nd number: 6
Sum = 11
```

Another way to convert the `string` returned by the `ReadLine` method to an `int` value is to use the `System.Int32.Parse` method or simply `int.Parse` as in the following example.

```
1  using System;
2
3  namespace Sum
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter 1st number: ");
10             int x = int.Parse(Console.ReadLine());
11
12             Console.Write("Enter 2nd number: ");
13             int y = System.Int32.Parse(Console.ReadLine());
14
15             int sum = x + y;
16
17             Console.Write("Sum = ");
18             Console.WriteLine(sum);
19         }
20     }
21 }
```

Click to run ►

In the above code, there is no different between `int.Parse` and `System.Int32.Parse`. `int.Parse` is just a shorthand. You can input values from different types as in the following examples:


```
1 bool b = bool.Parse(Console.ReadLine());  
2 char c = char.Parse(Console.ReadLine());  
3 ulong y = ulong.Parse(Console.ReadLine());  
4 double z = double.Parse(Console.ReadLine());
```


Chapter 4

Operators

Operators are used to perform operations on variables and values. In this lecture, you will learn about operators in C#.

4.1 Sign operators

A sign operator indicates whether a value is positive or negative.

Operator	Description	Example
+	Unary plus operator	+5
-	Unary minus operator	-7

4.2 Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations as shown in the table below.

Operator	Description	Example
+	Adds two values	$x + y$
-	Subtracts one value from another	$x - y$
*	Multiplies two values	$x * y$
/	Divides one value by another	x / y
%	Returns the division remainder	$x \% y$

The following example shows a program performs some mathematical operations and prints their outputs.

```
1  using System;
2  namespace ArithmeticOperations
3  {
4      class Program
5      {
6          static void Main()
7          {
8              int x = 5, y = 2;
9              Console.WriteLine(x / y); // Division
10             Console.WriteLine(x % y); // Modulus
11             Console.WriteLine((double)x / y); // Division
12         }
13     }
14 }
```

Click to run ►

In the above program, line 9 divides the value stored in `x` by the value stored in `y` (i.e. divides `5` by `2`) and prints the division result to the screen. Note that the division result is `2` not `2.5` because dividing an `int` value by an `int` value results in an `int` value. Line 10 uses the modulus `%` operator to calculate the division remainder of `x` by `y`, which is `1`.

Line 11 uses casting to convert the value stored in `x` to a `double` value before using the division operator `/` to divide the operand `(double)x` by the operand `y`. Note that the first operand is of type `double` while the second operand is of type `int`. Therefore, the compiler will implicitly convert the second operand (i.e. the value stored in `y`) to a `double` value to match the data type of the first operand (i.e. `(double)x`) before performing the division. The result of the division operation will be of type `double` because the two operands are now of type `double`. Therefore, the result will be `2.5`.

The operands of an arithmetic operation must be of the same type before the arithmetic operation can take place. The result of an arithmetic operation will have the same type as the operands.

If the operands of an arithmetic operation have different types, the compiler will try to implicitly convert one of the operands to the type of the other operand to make them of the same type before performing the arithmetic operation.

4.3 Increment and Decrement Operators

The increment operator `++` increases the value of a variable by `1`. Similarly, the decrement operator `--` decreases the value of a variable by `1`. However, there is an important difference when these two operators are used as a prefix and a postfix as shown in the following table.

Operator	Description	Example
Prefix ++	Increments the value before returning it	<code>++x</code>
Prefix --	Decrements the value before returning it	<code>--x</code>
Postfix ++	Increments the value after returning it	<code>x++</code>
Postfix --	Decrements the value after returning it	<code>x--</code>

If you use the `++` operator as a prefix like: `++x`, the value of `x` is incremented by `1` first; then the new value is returned. For example,

```
1 int x = 5;
2 int y = ++x;
```

In the above code, line 2 increments the value of the variable `x` by `1` making it `6` before assigning it to the variable `y`. Therefore, after executing line 2, the value of `x` will be `6` and the value of `y` will be `6` as well. This is equivalent to writing:

```
1  int x = 5;  
2  x = x + 1;  
3  int y = x;
```

If you use the `++` operator as a postfix like: `x++`, the original value of `x` is returned first; then `x` is incremented by `1`. For example,

```
1  int x = 5;  
2  int y = x++;
```

In the above code, line 2 assigns the value of the variable `x` to the variable `y` before it increments the value of the variable `x` by `1`. Therefore, after executing line 2, the value of `x` will be `6` but the value of `y` will be `5`. This is equivalent to writing:

```
1  int x = 5;  
2  int y = x;  
3  x = x + 1;
```

The `-` operator works in a similar way to the `++` operator in terms of prefix and postfix behavior but it decreases the value by `1`.

4.4 Assignment Operators

Assignment operators are used to assign values to variables. The assignment operator `=` can be used to assign a value to a variable. The following example assigns the value `10` to a variable named `x`.

```
1 x = 10;
```

There are other assignment operators as listed in the following table.

Operator	Example	Same As
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code>	<code>x /= 3</code>	<code>x = x / 3</code>
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>

The assignment operators in the above table are considered short-hands that allow a programmer to write code faster. For example:

```
1 // Instead of writing
2 // totalIncome = totalIncome + newIncome;
3 // A programmer can write
4 totalIncome += newIncome;
```


4.5 Comparison Operators

Comparison operators are used to compare two values. The following table lists the comparison operators.

Operator	Name	Example
<code>==</code>	Equal to	<code>x == y</code>
<code>!=</code>	Not equal	<code>x != y</code>
<code>></code>	Greater than	<code>x > y</code>
<code><</code>	Less than	<code>x < y</code>
<code>>=</code>	Greater than or equal to	<code>x >= y</code>
<code><=</code>	Less than or equal to	<code>x <= y</code>

The result of a comparison is a Boolean value. For example,

```
1  int x = 5;
2  int y = 6;
3  bool result = x > y;
```

In the above code, line 3 performs a comparison between the values of two variables `x` and `y`. Afterward it assigns the result of the comparison, which is `false`, to a variable named `result`. Note that the result of evaluating a comparison is always a Boolean value (i.e. `true` or `false`).

4.6 Logical Operators

Logical operators are useful to perform the logical operation on Boolean operands. C# provides the following three logical operators.

Operator	Name	Example
<code>&&</code>	Conditional logical AND operator	<code>a && b</code>
<code> </code>	Conditional logical OR operator	<code>a b</code>
<code>!</code>	Logical negation operator (NOT)	<code>!a</code>

The conditional logical AND operator `&&` returns `true` only if both operands are `true`. The conditional logical OR operator `||` returns `false` only if both operands are `false` as shown below.

Operand 1	Operand 2	AND	OR
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

The logical negation operator `!` returns `false` if the operand is `true` and vice versa as shown in the following table.

Operand	NOT
false	true
true	false

4.7 Expressions

There are two types of expressions that a computer can evaluate, which are **arithmetic expressions** and **logical expressions**. For example,

```
1 namespace Expressions
2 {
3     class Program
4     {
5         static void Main()
6         {
7             int x = 5, y = 6;
8             int a = x + y; // Arithmetic expression
9             bool b = x > y; // Logical (Boolean) expression
10            System.Console.WriteLine(a);
11            System.Console.WriteLine(b);
12        }
13    }
14 }
```

Click to run ►

In the above code, line 3 performs an arithmetic operation. It evaluates the **arithmetic expression** `x + y` and assigns its value (i.e. `11`) to the variable `a`. Note that the result of evaluating an **arithmetic expression** is always a numeric value. Line 4 performs a logical operation. It evaluates the **logical expression** `x > y` and assigns its value (i.e. `false`) to the variable `b`. Note that the result of evaluating a **logical expression** (also known as **Boolean expression**) is always a Boolean value (i.e. `true` or `false`).

4.8 Conditional Operator

The conditional operator `?:`, also known as the **ternary conditional operator**, evaluates a Boolean expression and returns the result of one of the two expressions, depending on whether the Boolean expression evaluates to `true` or `false`. The syntax for the conditional operator is as follows:

Syntax

```
condition ? expression1 : expression2
```

If the `condition` is `true`, the conditional operator returns the value of `expression1`. If the `condition` is `false`, the conditional operator returns the value of `expression2`. For example,

```
1 int score = 90;
2 string result = score >= 50 ? "Pass" : "Fail";
3 System.Console.WriteLine(result); // Output: Pass
```

In the above example, line 2 uses the conditional operator `?:` to evaluate the condition `score >= 50` before returning one of the two expressions (i.e. `"Pass"` or `"Fail"`) based on the value of the condition. Clearly, the value of the condition is `true`. Therefore, the conditional operator `?:` returns the value of the first expression (i.e. `"Pass"`), which is assigned to the `result` variable.

4.9 String Concatenation

Concatenation is the process of appending one string to the end of another string. You concatenate strings by using the `+` operator. For example,

```
1 namespace Concatenation
2 {
3     class Program
4     {
5         static void Main()
6         {
7             string firstName = "Mohamed ";
8             string lastName = "Handosa";
9             string fullName = firstName + lastName;
10            System.Console.WriteLine(fullName);
11        }
12    }
13 }
```

Click to run ►

In the above example, line 9 uses the `+` operator to concatenate the two string values stored in the two variables `firstName` and `lastName` before storing the result in a variable named `fullName`. The output should be:

Output

MohamedHandosa

4.10 Escape Sequences

Escape sequences are used to represent certain special characters within string literals and character literals. The following table shows some special characters along with their escape sequences.

Special Character	Escape Sequence
Single quote <code>'</code>	<code>\'</code>
Double quote <code>"</code>	<code>\"</code>
Backslash <code>\</code>	<code>\\</code>
New line	<code>\n</code>
Tab	<code>\t</code>

The following example uses escape sequences to represent special characters within string literals and character literals.

```
1 char ch = '\\';
2 string text = "Name:\t\"Mohamed\"\\nAddress:\t\"'Egypt'";
3 System.Console.WriteLine(ch);
4 System.Console.WriteLine(text);
```

Output

```
\
Name:  "Mohamed"
Address:  'Egypt'
```

4.11 Operator Precedence

In an expression with multiple operators, the operators with higher precedence are evaluated before the operators with lower precedence.

In the following example, the multiplication is performed first because it has higher precedence than addition:

```
1 int a = 2 + 2 * 2;  
2 System.Console.WriteLine(a);
```

Output

6

Parentheses can change the order of evaluation imposed by operator precedence. For example,

```
1 int a = (2 + 2) * 2;  
2 System.Console.WriteLine(a);
```

Output

8

The following table lists the C# operators starting with the highest precedence to the lowest. The operators within each row have the same precedence.

Operator	Category
<code>x++</code> <code>x--</code>	Unary
<code>+x</code> <code>-x</code> <code>!x</code> <code>++x</code> <code>--x</code>	Unary
<code>x * y</code> <code>x / y</code> <code>x % y</code>	Multiplicative
<code>x + y</code> <code>x - y</code>	Additive
<code>x < y</code> <code>x > y</code> <code>x <= y</code> <code>x >= y</code>	Relational
<code>x == y</code> <code>x != y</code>	Equality
<code>x && y</code>	Conditional AND
<code>x y</code>	Conditional OR
<code>c ? t : f</code>	Conditional operator
<code>x = y</code> <code>x += y</code> <code>x -= y</code> <code>x *= y</code> <code>x /= y</code> <code>x %= y</code>	Assignment

4.12 Operator Associativity

When operators have the same precedence, associativity of the operators determines the order in which the operations are performed:

- **Left-associative operators** are evaluated in order from left to right (e.g., `a + b - c` is evaluated as `(a + b) - c`).
- **Right-associative operators** are evaluated in order from right to left. (e.g., `x = y = z` is evaluated as `x = (y = z)`).

The following table lists operators along with their associativity.

Operator	Associativity
<code>x++</code> <code>x--</code>	Left to right
<code>+x</code> <code>-x</code> <code>!x</code> <code>++x</code> <code>--x</code>	Right to left
<code>x * y</code> <code>x / y</code> <code>x % y</code>	Left to right
<code>x + y</code> <code>x - y</code>	Left to right
<code>x < y</code> <code>x > y</code> <code>x <= y</code> <code>x >= y</code>	Left to right
<code>x == y</code> <code>x != y</code>	Left to right
<code>x && y</code>	Left to right
<code>x y</code>	Left to right
<code>c ? t : f</code>	Right to left
<code>x = y</code> <code>x += y</code> <code>x -= y</code> <code>x *= y</code> <code>x /= y</code> <code>x %= y</code>	Right to left

Use parentheses to change the order of evaluation. For example,

```
1  int a = 13 / 5 / 2;    // a = 1
2  int b = 13 / (5 / 2); // b = 6
```

In the above example, line 1 divides `13` by `5` then divides the result (i.e. `2`) by `2`. Line 2 divides `5` by `2` first then divides `13` by the result of `5 / 2` (i.e. `2`).

4.13 Operand Evaluation

Unrelated to operator precedence and associativity, operands in an expression are evaluated from left to right. The following examples demonstrate the order in which operators and operands are evaluated:

Expression	Order of Evaluation
<code>a + b</code>	<code>a, b, +</code>
<code>a + b * c</code>	<code>a, b, c, *, +</code>
<code>a / b + c * d</code>	<code>a, b, /, c, d, *, +</code>
<code>a / (b + c) * d</code>	<code>a, b, c, +, /, d, *</code>

Typically, all operator operands are evaluated. However, some operators evaluate operands conditionally. That is, the value of the left-most operand defines if (or which) other operands should be evaluated. These operators are the conditional logical AND `&&` and OR `||` operators and the conditional operator `?:`. For example,

```

1  int x = 10, y = 5;
2  bool a = x < y && y == 5;      // a = false
3  bool b = x > y || x == 7;      // b = true
4  int z = x < y ? x * y : x / y;  // z = 2

```

In the above code, line 2 evaluates `x < y` and finds that it is `false`. Therefore, no need to evaluate `y == 5` because anyways the result of `&&` will be `false`.

Similarly, line 3 evaluates `x > y` and finds that it is `true`. Therefore, no need to evaluate `x == 7` because anyways the result of `||` will be `true`.

Line 4 evaluates `x < y` and finds that it is `false`. Therefore, the value of the expression `x < y ? x * y : x / y` will be the value of the second operand (i.e. `x / y`) and there is no need to evaluate the first operand (i.e. `x * y`).

Chapter 5

Branching

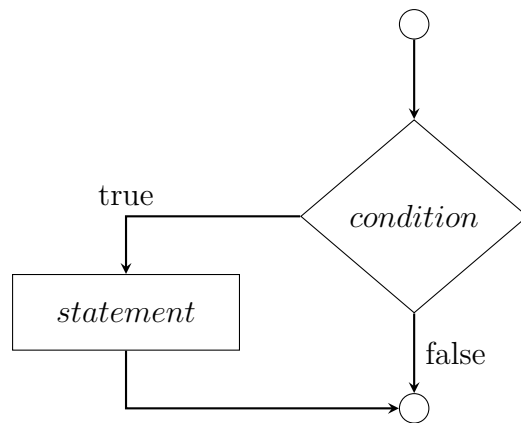
When an “Algorithm” makes a choice to do one of two (or more) things this is called branching. C# provides three programming statements that can be used to branch. These statements are the `if` statement, the `if..else` statement, and the `switch` statement.

5.1 The `if` Statement

An `if` statement specifies a single statement (or a block of statements) that will be executed only if a condition is `true`. The syntax of an `if` statement takes the following form:

Syntax

```
if(condition) statement
```



To specify a block of statements to be executed if a condition is `true`, the syntax can take the following form:

Syntax

```
if(condition)  
{  
    statement1  
    statement2  
    ...  
    statementn  
}
```

The following program prints `Pass` to the screen only if the value of the entered score is greater than or equal to `50`.

```
1  using System;
2
3  namespace Score2Grade
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter score: ");
10             int score = int.Parse(Console.ReadLine());
11
12             if (score >= 50)
13             {
14                 Console.WriteLine("Pass");
15             }
16
17             Console.WriteLine("End of program");
18         }
19     }
20 }
```

Click to run ►

If the above example, lines 12-15 constitute an `if` statement. Executing this statement starts by evaluating its condition, which is the logical expression `score >= 50`. If the condition value is `true`, the block (i.e. lines 13-15) will execute. Otherwise, the block will be ignored and the execution will jump to the statement after the `if` statement (i.e. the statement at line 17).

Assuming that the user enters `90` as input, the output will be:

Output

```
Enter score: 90
Pass
End of program
```

Assuming that the user enters `30` as input, the output will be:

Output

```
Enter score: 30
End of program
```

5.2 The `if..else` Statement

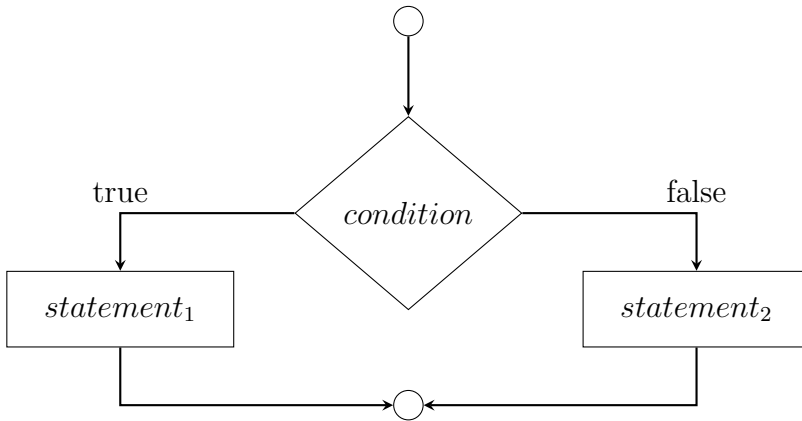
The `if..else` statement is an `if` statement with an `else` part. The `if` part specifies a statement (or block) to be executed if the condition is `true` while the `else` part specifies a statement (or block) to be executed if the condition is `false`. The syntax of an `if..else` statement takes the following form:

Syntax

```
if(condition) statement1 else statement2
```

Where `statement1` will execute only if the `condition` is `true`

and `statement2` will execute only if the `condition` is `false`.



The `if..else` statement can specify statement blocks instead of single statements. The syntax can take the following form:

Syntax

```
if(condition)
{
    // The code block to be executed if the condition is true
}
else
{
    // The code block to be executed if the condition is false
}
```

The following program prints `Pass` to the screen only if the value of the entered score is greater than or equal to `50`. Otherwise, it prints `Fail` to the screen.

```
1  using System;
2
3  namespace Score2Grade
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter score: ");
10             int score = int.Parse(Console.ReadLine());
11
12             if (score >= 50)
13             {
14                 Console.WriteLine("Pass");
15             }
16             else
17             {
18                 Console.WriteLine("Fail");
19             }
20
21             Console.WriteLine("End of program");
22         }
23     }
24 }
```

Click to run ►

If the above example, lines 12-19 constitute an `if..else` statement. Executing this statement starts by evaluating its condition,

which is the logical expression `score >= 50`. If the condition value is `true`, the block specified by the `if` part (i.e. lines 13-15) will execute. Otherwise, the block specified by the `else` part (i.e. lines 17-19) will execute. In either case, after executing the `if..else` statement, the execution proceeds to the next statement (i.e. the statement at line 21).

Assuming that the user enters `90` as input, the output will be:

```
Output
Enter score: 90
Pass
End of program
```

Assuming that the user enters `30` as input, the output will be:

```
Output
Enter score: 30
Fail
End of program
```

When a program encounters an `if..else` statement, it will execute the `if` branch or the `else` branch based on the value of the `condition`. A program can never execute both branches.

5.3 Nested if Statements

It is always legal in C# to nest `if` statements, which means you can use one `if` or `if..else` statement inside another `if` or `if..else` statement. This can be a little tricky but usually helps writing better programs. For example, suppose that we need to develop a C# program that takes a score as input and outputs the corresponding grade. One way to write this program is as follows:

```
1  using System;
2
3  namespace Score2Grade
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.WriteLine("Enter score: ");
10             int score = int.Parse(Console.ReadLine());
11
12             if (score >= 85)
13             {
14                 Console.WriteLine( "Excellent");
15             }
16
17             if (score < 85 && score >= 75)
18             {
19                 Console.WriteLine( "Very Good");
20             }
21
```

Click to run ►

```
22         if (score < 75 && score >= 65)
23         {
24             Console.WriteLine( "Good");
25         }
26
27         if (score < 65 && score >= 50)
28         {
29             Console.WriteLine( "Pass");
30         }
31
32         if (score < 50)
33         {
34             Console.WriteLine( "Fail");
35         }
36     }
37 }
38 }
```

In the above example, lines 12-15 use an if statement to print the text `Excellent` to the screen only if the `score` is greater than or equal to `85`. Lines 17-20 use an if statement to print `Very Good` to the screen only if the `score` is less than `85` and greater than or equal to `75`. Lines 22-25 use an if statement to print `Good` to the screen only if the `score` is less than `75` and greater than or equal to `65`. Lines 27-30 use an if statement to print `Pass` to the screen only if the `score` is less than `65` and greater than or equal to `50`. Lines 32-35 use an if statement to print `Fail` to the screen only if the `score` is less than `50`.

The way the above program is written has two drawbacks. First, the program will go through all of the `if` statements, evaluating their conditions, regardless of the `score` value entered by the user. For example, suppose that the user enters a score value of `90`. The program will evaluate the condition `score >= 85` and finds that its value is `true`. Therefore, the program will print the text `Excellent` to the screen. At this point, we have obtained the required result and we should be done. However, the program will not stop at this point but rather will proceed to the next `if` statements, evaluating their conditions. This is a waste of time and processing power. Second, the condition of some `if` statements consists of two comparisons. Evaluating such condition requires performing two comparison operations besides a conditional AND operation.

The following is a better version of the previous program that uses nested `if..else` statements instead, which reduces the number of required comparisons and avoids the use of AND operations.

```
1  using System;
2
3  namespace Score2Grade
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.WriteLine("Enter score: ");
10             int score = int.Parse(Console.ReadLine());
```

Click to run ►

```
11
12     if (score >= 85)
13     {
14         Console.WriteLine( "Excellent");
15     }
16     else // score < 85
17     {
18         if (score >= 75)
19         {
20             Console.WriteLine( "Very Good");
21         }
22         else // score < 75
23         {
24             if (score >= 65)
25             {
26                 Console.WriteLine( "Good");
27             }
28             else // score < 65
29             {
30                 if (score >= 50)
31                 {
32                     Console.WriteLine( "Pass");
33                 }
34                 else // score < 50
35                 {
36                     Console.WriteLine( "Fail");
37                 }
38             }
39         }
40     }
41 }
42
43 }
```

5.4 The `switch` Statement

The `switch` statement selects a statement list to execute based on a pattern match with a match expression. The syntax takes the form:

Syntax

```
switch(expression)
{
    case x:
        // code block
        break;
    case y:
        // code block
        break;
    default:
        // code block
        break;
}
```

In order to execute a `switch` statement, the `expression` is evaluated once. Afterward, the value of the `expression` is compared to the value of each `case` one by one. If there is a match between the value of the `expression` and the value of a `case`, the associated code block is executed before executing the `break` statement to exit the `switch` statement. If no `case` value matches the value of the `expression`, the `default` code block is executed.

The following program takes the weekday number as input and outputs the the weekday name:

```
1  using System;
2
3  namespace WeekdayNumber2Name
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter a weekday number: ");
10
11              int day = int.Parse(Console.ReadLine());
12
13              switch (day)
14              {
15                  case 1:
16                      Console.WriteLine("Saturday");
17                      break;
18
19                  case 2:
20                      Console.WriteLine("Sunday");
21                      break;
22
23                  case 3:
24                      Console.WriteLine("Monday");
25                      break;
26
27                  case 4:
28                      Console.WriteLine("Tuesday");
29                      break;
30
```

Click to run ►

```
31         case 5:
32             Console.WriteLine("Wednesday");
33             break;
34
35         case 6:
36             Console.WriteLine("Thursday");
37             break;
38
39         case 7:
40             Console.WriteLine("Friday");
41             break;
42     }
43 }
44 }
45 }
```

In the above example, assuming that the user enters the value `6` as input, the output will be

Output

Thursday

5.4.1 The `break` Statement

Within a `switch` statement, control cannot fall through from one `case` section to the next. Therefore, we use the `break` statement at the end of each `case` section to break out of the `switch` block.

5.4.2 The default Case

The `default` case is optional. It specifies statements to execute when the `expression` of the `switch` statement doesn't match any other `case`. For example,

```
1  using System;
2
3  namespace TrafficLights
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.Write("Enter a traffic light color: ");
10             string color = Console.ReadLine();
11
12             switch (color)
13             {
14                 case "red":
15                     Console.WriteLine("Stop");
16                     break;
17
18                 case "green":
19                     Console.WriteLine("Go");
20                     break;
21
22                 case "yellow":
23                     Console.WriteLine("Caution");
24                     break;
25             }
```

Click to run ►

```
26         default:
27             Console.WriteLine("Invalid color");
28             break;
29     }
30 }
31 }
32 }
```

In the above example, lines 26-28 specify a `default` case that will execute only if the `color` expression does not match any of the other cases (i.e. `red`, `green`, and `yellow`). Assuming that a user enters `blue` as input, the output of the program will be:

Output

```
Enter a traffic light color:  blue
Invalid color.
```

5.5 Scope of Variables

The **scope** of a variable is the part of the program where that variable is accessible (i.e. its value can be read and/or changed). A variable is not accessible outside of its scope. A variable that is defined within a block of code is accessible only within that block of code. For example,

```
1  using System;
2
3  namespace ScopeOfVariables
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.WriteLine(x); // Error
10             Console.WriteLine(y); // Error
11
12             int x = 5; // Here, the scope of x starts
13
14             Console.WriteLine(x); // Ok
15             Console.WriteLine(y); // Error
16
17             if(x >= 5)
18             {
19                 Console.WriteLine(x); // Ok
20                 Console.WriteLine(y); // Error
21
22                 int y = 6; // Here, the scope of y starts
23
24                 Console.WriteLine(x); // Ok
25                 Console.WriteLine(y); // Ok
26             } // Here, the scope of y ends
27
28             Console.WriteLine(x); // Ok
29             Console.WriteLine(y); // Error
30         } // Here, the scope of x ends
31     }
32 }
```

Click to run ►

In the above code, line 12 defines a variable named `x` within the block that starts with `{` at line 8 and ends with `}` at line 30. The variable `x` cannot be accessed outside this block and of course cannot be accessed before it is defined. Therefore, line 9 cannot access the variable `x`. Meanwhile, lines 14, 19, 24, and 28 can access the variable `x`.

Similarly, line 22 defines a variable named `y` within the block that starts with `{` at line 18 and ends with `}` at line 26. The variable `y` cannot be accessed outside this block and of course cannot be accessed before it is defined. Therefore, lines 10, 15, and 20 cannot access the variable `y`. Meanwhile, line 25 can access the variable `y`. Note that line 29 cannot access the variable `y` because the scope of `y` ends at line 26.

Chapter 6

Looping

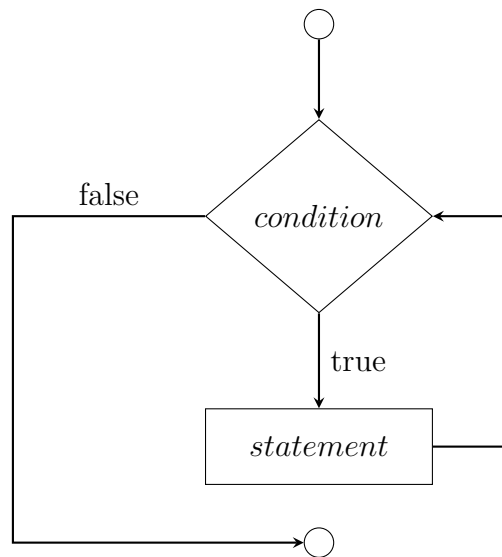
An `if` statement executed a statement (or block) **once** if a specified condition is `true`. In contrast, a loop statement executes a statement (or block) **repeatedly** as long as a specified condition is `true`. This allows for executing a statement (or block) multiple times. Looping can use a `while`, a `do..while`, or a `for` loop.

6.1 The while Loop

The `while` loop statement loops through a a statement or a block of statements as long as a specified condition is `true`. The syntax of a `while` statement takes the following form:

Syntax

```
while (condition) statement
```



To specify a block of statements to be executed as long as a condition is `true`, the syntax can take the following form:

Syntax

```
while (condition)  
{  
    // The code block to be executed  
}
```


The following example uses a `while` loop to print the values `1` through `5` to the screen.

```
1  using System;
2
3  namespace LoopingUsingWhile
4  {
5      class Program
6      {
7          static void Main()
8          {
9              int counter = 1;
10
11              while(counter <= 5)
12              {
13                  Console.Write(counter + " ");
14                  counter++;
15              }
16          }
17      }
18  }
```

Click to run ►

In the above example, the block starting at line 12 and ending at line 15 will execute, over and over again, as long as the condition `counter <= 5` is `true`. The output should be:

Output

1 2 3 4 5

Note that line 14 increases the value of `counter` by `1`, every time the `while` block (i.e. lines 12-15) is executed. Therefore, after executing the `while` block five times, the value of `counter` will be `6` and the `while` condition (i.e. `counter <= 5`) will be `false`, which results in exiting the `while` loop.

An **infinite loop** is a loop whose condition remains `true` forever and never changes to `false`. This loop will repeatedly execute its statement (or block) forever and will never end!

The following program has an infinite loop. It will print `1` to the screen over and over again, forever.

```
1 namespace InfiniteLoop
2 {
3     class Program
4     {
5         static void Main()
6         {
7             int counter = 1;
8
9             while(counter <= 5)
10             {
11                 System.Console.Write(counter + " ");
12             }
13         }
14     }
15 }
```

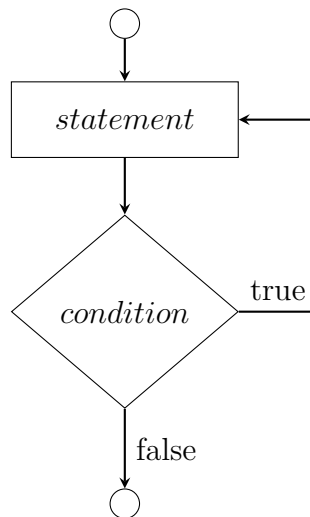
Click to run ►

6.2 The `do..while` Loop

The `do..while` loop is a variant of the `while` loop. This loop will execute its statement (or block) once, before checking if the condition is `true`, then it will execute its statement (or block) repeatedly as long as the condition is `true`. The syntax of a `do..while` statement takes the following form:

Syntax

```
do statement while (condition);
```



To specify a block of statements to be executed as long as a condition is `true`, the syntax can take the following form:

Syntax

```
do
{
    // The code block to be executed
}
while (condition);
```

The following example uses a `do..while` loop to verify user input.

```
1  using System;
2  namespace VerifyUserInput
3  {
4      class Program
5      {
6          static void Main()
7          {
8              int score;
9
10             do
11             {
12                 Console.Write("Enter score (0 : 100): ");
13                 score = int.Parse(Console.ReadLine());
14             }
15             while(score < 0 || score > 100);
16
17             Console.WriteLine("Score accepted!");
18         }
19     }
20 }
```

Click to run ►

In the above example, the `do..while` block (lines 11-14) will be executed first. Afterward, the `do..while` condition will be tested. If the condition is `true`, the `do..while` block will be executed again. If the condition is `false`, execution will exit from the `do..while` loop and proceed to the next statement at line 17.

Note that the variable named `score` is defined at line 8 before the `do..while` loop and not inside its block. This is necessary because the `do..while` condition at line 15 needs to access the `score` variable. For example, if the `score` variable is defined by mistake at line 13 instead of line 8, its scope will end at line 14 and the program will not be able to access it at line 15, where it evaluates the condition.

Assuming that the user will enter the values `-10`, `250`, and `99`, respectively, the output of the above program will be:

Output

```
Enter score (0 : 100): -10
Enter score (0 : 100): 250
Enter score (0 : 100): 99
Score accepted!
```

The statement (or block) of a `do..while` loop will always be executed at least once, even if the condition is `false`, because the statement (or block) is executed before the condition is tested.

A program with a `do..while` loop can be rewritten using a `while` loop. For example, the following program uses a `while` loop instead of a `do..while` loop to verify user input.

```
1  using System;
2  namespace VerifyUserInput
3  {
4      class Program
5      {
6          static void Main()
7          {
8              Console.WriteLine("Enter score (0 : 100): ");
9              int score = int.Parse(Console.ReadLine());
10
11              while (score < 0 || score > 100)
12              {
13                  Console.WriteLine("Enter score (0 : 100): ");
14                  score = int.Parse(Console.ReadLine());
15              }
16
17              Console.WriteLine("Score accepted!");
18          }
19      }
20 }
```

Click to run ►

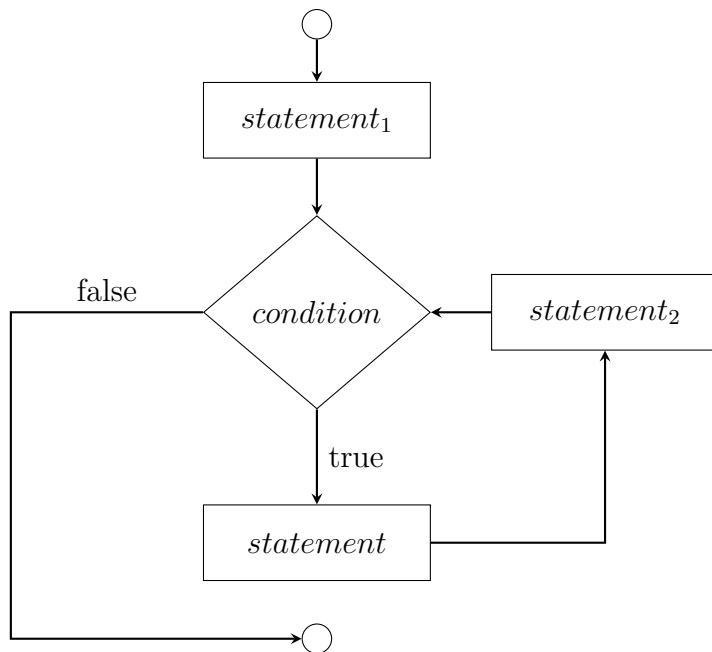
In the above code, note that the code for user input is written twice. First, at lines 8 and 9 before the `while` loop. Second, at lines 13 and 14 within the block of the `while` loop. This is to ensure that the code for user input will execute at least once. Using `do..while` in such cases saves you from writing the same code multiple times.

6.3 The for Loop

The `for` loop can be used when we know exactly the number of times we want to execute a statement (or block). The syntax of a `for` loop statement takes the following form:

Syntax

```
for (statement1; condition; statement2) statement
```



Where *statement*₁ is executed only one time at the beginning of the

`for` loop. Afterward, the `condition` is checked. If the condition is `false`, the execution exits from the `for` loop. If the condition is `true`, `statement` is executed and `statement2` is executed then the `condition` is checked again and so on.

To specify a block of statements to be executed, the syntax can be:

Syntax

```
for (statement1; condition; statement2)  
{  
    // The code block to be executed  
}
```

The following program prints the values `1` through `5` to the screen.

```
1 namespace LoopingUsingFor  
2 {  
3     class Program  
4     {  
5         static void Main()  
6         {  
7             for (int x = 1; x <= 5; x++)  
8                 System.Console.Write(x + " ");  
9         }  
10    }  
11 }
```

Click to run ►

In the above program, the `int x = 1` statement at line 7 is executed

only once at the beginning of the `for` loop. Now, the value of `x` is `1`. Afterward, the condition `x <= 5` is checked and is found to be `true`. Therefore, the statement at line 8 is executed, which prints the current value of `x` (i.e. `1`) to the screen, then the `x++` statement at line 7 is executed, which increments the value of `x` to be `2`. Afterward, the condition `x <= 5` is checked again. This will continue until the condition becomes `false`. The output of this program should be:

Output
1 2 3 4 5

The following program prints the odd values between `1` and `10`:

```
1 namespace OddNumbers
2 {
3     class Program
4     {
5         static void Main()
6         {
7             for (int x = 1; x < 10; x += 2)
8             {
9                 System.Console.Write(x + " ");
10            }
11        }
12    }
13 }
```

Click to run ►

In the above program, the value of `x` is incremented by `2` after each iteration. The output of this program should be:

Output

1 3 5 7 9

The following program takes five numbers from the user and prints their sum to the screen.

```
1  using System;
2  namespace Add5Numbers
3  {
4      class Program
5      {
6          static void Main()
7          {
8              float sum = 0;
9
10             for (int x = 1; x <= 5; x++)
11             {
12                 Console.Write("Number (" + x + "): ");
13                 float n = float.Parse(Console.ReadLine());
14                 sum += n;
15             }
16
17             Console.WriteLine("\nSum = " + sum);
18         }
19     }
20 }
```

Click to run ►

In the above program, line 8 defines a variable named `sum` and set its initial value to `0`. The `for` loop defined in lines 10-15 will iterate five times before it terminates when the value of `x` becomes `6` (i.e. the condition value becomes `false`). In each iteration,

- Line 12 asks the user to input a number.
- Line 13 takes the input and stores it in a variable `n`.
- Line 14 adds the value stored in `n` to the `sum` variable.

Note that the `sum` variable acts as a bucket that we add to with each iteration of the `for` loop. This why its initial value is set to `0`. After executing the five iterations of the `for` loop, the execution proceeds to line 17 and prints the value of `sum` to the screen.

Assuming that the numbers entered by the user are `2.5`, `15.25`, `3`, `8.1`, and `170`, the output of the program should be:

Output

```
Number (1):  2.5
Number (2):  15.25
Number (3):  3
Number (4):  8.1
Number (5):  170

Sum = 198.85
```

A program with a `for` loop can be rewritten using a `while` loop.

For example, the following program uses a `while` loop instead of a `for` loop to take five numbers from the user and print their sum to the screen.

```
1  using System;
2  namespace Add5Numbers
3  {
4      class Program
5      {
6          static void Main()
7          {
8              float sum = 0;
9
10             int x = 1;
11             while (x <= 5)
12             {
13                 Console.WriteLine("Number (" + x + "): ");
14                 float n = float.Parse(Console.ReadLine());
15                 sum += n;
16                 x++;
17             }
18
19             Console.WriteLine("\nSum = " + sum);
20         }
21     }
22 }
```

Click to run ►

In the above code, we just placed the statement `int x = 1` above the loop and placed the `x++` statement at the end of the loop block.

6.4 The break Statement

The `break` statement can be used to break out of a `switch` statement as discussed in section 5.4. The `break` statement can also be used to break out of a `while`, a `do..while`, or a `for` loop. When the `break` statement is executed the execution jumps out of the loop and proceeds to the statement after the loop. For example,

```
1  using System;
2
3  namespace AddUpto5Numbers
4  {
5      class Program
6      {
7          static void Main()
8          {
9              float sum = 0;
10
11              for (int x = 1; x <= 5; x++)
12              {
13                  Console.Write("Number (" + x + "): ");
14                  float n = float.Parse(Console.ReadLine());
15                  if (n == 0) break;
16                  sum += n;
17              }
18
19              Console.WriteLine("\nSum = " + sum);
20          }
21      }
22  }
```

Click to run ►

The above program takes up to five numbers from the user and prints their sum to the screen. The user can stop entering more numbers at any point by simply entering a `0`. Line 15 breaks out of the loop if the condition `n == 0` is `true` (i.e. if the user enters a `0`).

Assuming that the numbers entered by the user are `2.5`, `15.25`, `3`, and `0`, the output of the program should be:

Output

```
Number (1):  2.5
Number (2):  15.25
Number (3):  3
Number (4):  0

Sum = 20.75
```

There is no point in having a loop with a `break` statement that will always execute and break out of the loop from the very first iteration. If we really want to always break out of a given loop, then why would we create that loop at the first place?

It makes more sense that we would want to break out of a loop only if some condition holds. Therefore, a `break` statement does not come by itself but rather comes within an `if` statement that controls whether or not the `break` statement should execute.

6.5 The continue Statement

The `continue` statement breaks out of the current iteration of a loop, and continues with the next iteration.

The following program takes five numbers from the user, ignores the negative numbers, and prints the sum of the positive numbers.

```
1  using System;
2
3  namespace AddPositiveNumbers
4  {
5      class Program
6      {
7          static void Main()
8          {
9              float sum = 0;
10
11              for (int x = 1; x <= 5; x++)
12              {
13                  Console.Write("Number (" + x + "): ");
14                  float n = float.Parse(Console.ReadLine());
15                  if (n <= 0) continue;
16                  sum += n;
17              }
18
19              Console.WriteLine("\nSum = " + sum);
20          }
21      }
22  }
```

Click to run ►

In the above program, line 15 breaks out of the current loop iteration if the condition `n <= 0` is `true` (i.e. if the user enters a `0` or a negative number). Line 16 adds the value entered by the user to the `sum` variable. Note that line 16 will execute only if the `continue` statement does not execute (i.e. if the user enters a positive number).

Assuming that the numbers entered by the user are `2.5`, `-15.25`, `3`, `-8.1`, and `170`, the output of the program should be:

Output

```
Number (1):  2.5
Number (2): -15.25
Number (3):  3
Number (4): -8.1
Number (5): 170
```

```
Sum = 175.5
```

There is no point in having a loop with a `continue` statement that will always execute and break out of the current iteration. This way, any code written after `continue` will never execute.

It makes more sense that we would want to break out of the current iteration only if some condition holds. Therefore, a `continue` statement comes within an `if` statement.

6.6 Nested Loops

A **nested loop** is a loop within a loop, an inner loop within the body of an outer one. The first pass of the outer loop triggers the inner loop, which executes to completion. Then the second pass of the outer loop triggers the inner loop again. This repeats until the outer loop finishes.

For example, the following program prints the letters A through E 10 times in 10 different lines:

```
1  using System;
2
3  namespace NestedLoops
4  {
5      class Program
6      {
7          static void Main()
8          {
9              for (int line = 1; line <= 10; line++)
10             {
11                 for (char c = 'A'; c <= 'E'; c++)
12                 {
13                     Console.Write(c + " ");
14                 }
15
16                 Console.WriteLine();
17             }
18         }
19     }
20 }
```

Click to run ►

In the above program, there are two nested `for` loops. The outer loop is defined in lines 9-17 while the inner loop is defined in lines 11-14. The outer loop iterates 10 times and for each iteration,

- The inner loop is executed.
- Line 16 is executed which prints a new line to the screen.

Therefore, the inner loop itself will be executed 10 times. A single execution of the inner loop will iterate over the characters `A` through `E` printing each of them to the screen. The output should be:

Output

```
A B C D E
A B C D E
A B C D E
A B C D E
A B C D E
A B C D E
A B C D E
A B C D E
A B C D E
A B C D E
```

To practice on using nested loops, we can use them to print different shapes using asterisks `*`. For example, the following program prints a 5×5 square of asterisks to the screen.

```
1  using System;
2
3  namespace Square5x5
4  {
5      class Program
6      {
7          static void Main()
8          {
9              for (int row = 1; row <= 5; row++)
10             {
11                 for (int column = 0; column < 5; column++)
12                 {
13                     Console.Write('*');
14                 }
15
16                 Console.WriteLine();
17             }
18         }
19     }
20 }
```

Click to run ►

The output of the above program should be:

Output

```
*****
*****
*****
*****
*****
```

Another example is the following program that prints a right triangle:

```
1  using System;
2
3  namespace RightTriangle
4  {
5      class Program
6      {
7          static void Main()
8          {
9              for (int row = 1; row <= 5; row++)
10             {
11                 for (int col = 1; col <= row; col++)
12                 {
13                     Console.Write('*');
14                 }
15
16                 Console.WriteLine();
17             }
18         }
19     }
20 }
```

Click to run ►

Output

```
*
**
***
****
*****
```

The following program prints a flipped right triangle:

```
1  using System;
2
3  namespace FlippedRightTriangle
4  {
5      class Program
6      {
7          static void Main()
8          {
9              for (int row = 5; row >= 1; row--)
10             {
11                 for (int col = 1; col <= row; col++)
12                 {
13                     Console.Write('*');
14                 }
15
16                 Console.WriteLine();
17             }
18         }
19     }
20 }
```

Click to run ►

Output

```
*****
****
***
**
*
```

The following program prints all prime numbers between 1 and 30:

```
1  using System;                                     Click to run ►
2
3  namespace PrimeNumbers
4  {
5      class Program
6      {
7          static void Main()
8          {
9              for (int n = 2; n <= 30; n++)
10             {
11                 bool isPrime = true; // assume n is prime
12                 int max = n / 2; // Avoid unnecessary tests
13
14                 // Try dividing n by numbers from 2 to n/2
15                 for (int x = 2; x < max; x++)
16                 {
17                     // Test if n can be divided by x?
18                     if (n % x == 0)
19                     {
20                         // Yes, n can be divided by x
21                         isPrime = false; // n has failed
22                         break; // No need for more tests
23                     }
24                 }
25
26                 if (isPrime) Console.Write(n + " ");
27             }
28         }
29     }
30 }
```

In the above program, the outer loop in lines 9-27 iterates from `2` to `30`. In each iteration, line 11 sets `isPrime` to `true` because we start by assuming that the number `n` is prime.

The inner loop in lines 15-24 iterates from `2` to `n/2`. In each iteration, the `if` statement in lines 18-23 checks if `n` can be divided by `x`. If `n` can be divided by `x`, the number is not prime and we set `isPrime` to `false` before breaking out of the inner loop.

Line 26 prints the number `n` only if `isPrime` is still `true` after performing all the division tests, which means that `n` did not fail any of the tests and therefore `n` is prime. The output should be:

Output

```
2 3 4 5 7 11 13 17 19 23 29
```


Chapter 7

Arrays

An **array** is a data structure that can store multiple values of the same type. You can create an array by specifying the type and number of its elements. The number and type of array elements are fixed and cannot be changed after creating the array.

In C#, the `new` operator can be used to create an array. It creates the array in memory and returns a reference to it. A reference to an array is simply its address in memory. You need that reference to access the elements of the array. Therefore, you should create a variable to store that reference once it is returned by the `new` operator. Later, you can use this variable to access the elements of the array.

7.1 Creating Arrays

Obtaining an array is a two-step process. First, you must declare a **reference-type** variable of the desired array type. Second, you must create the array itself using the `new` operator and assign the returned reference to the **reference-type** variable created in the first step. For example, the following code line defines a variable named `x` of type `int[]` that can hold a reference to an array of `int` elements.

```
1 int[] x;
```

Now, we can use the `new` operator to create the array and return its reference. For example, the following code line creates an array containing 10 elements, where each element is of type `int`, and assigns the reference of the array to the `x` variable.

```
1 x = new int[10];
```

Note that `x` is a **reference-type** variable (see subsection 3.8.2). It does not hold the array itself but rather holds a reference to the array (i.e. it holds the address of the array in memory). The two code lines above can be merged into one code line as follows:

```
1 int[] x = new int[10];
```

7.2 Accessing Array Elements

Array elements can be accessed by their index. Each element in the array has an associated index. The **index** is a number placed inside square brackets which follow the array name.

Array index starts at `0`. The first element in an array is at index `0`, the second element in an array is at index `1`, and so on.

If the total number of elements in an array is `n`, then the index of the last element in that array is `n-1`.

For example, the following code snippet creates an array of 10 elements, each of type `int`, and assigns a value of `5` to the forth element in that array.

```
1  int[] x = new int[10];  
2  x[3] = 5; // Forth element is at index 3
```

In the above code, line 1 creates an `int` array of size `10` and assigns its reference to a variable named `x`. Line 2 uses the `x` variable to locate the array in memory before indexing into the array using a value of `3` to access the forth element in the array and assign a value of `5` to it.

7.3 Array Literal

In C#, an array literal is a comma-separated list of zero or more expressions, each of which represents an array element, enclosed in curly braces `{ }`. When you create an array using an array literal, it is initialized with the specified values as its elements, and its length is set to the number of specified values. For example,

```
1  using System;
2
3  namespace Cars
4  {
5      class Program
6      {
7          static void Main()
8          {
9              string[] cars = { "Fiat", "Nissan", "BMW" };
10
11              Console.WriteLine("cars[0]: " + cars[0]);
12              Console.WriteLine("cars[1]: " + cars[1]);
13              Console.WriteLine("cars(2): " + cars[2]);
14
15              Console.WriteLine("\nLength = " + cars.Length);
16          }
17      }
18  }
```

Click to run ►

In the above program, line 9 creates an array using an array literal and assigns its reference to a variable named `cars`. The created array is

of type `string` and has `3` elements. Lines 11-13 print the values of the array elements to the screen. Line 15 prints the length of the array to the screen. The output should be:

Output

```
cars[0]:  Fiat
cars[1]:  Nissan
cars(2):  BMW

Length = 3
```

7.4 Arrays and Loops

An array allows for storing multiple values in its elements. However, writing a piece of code to access a single element and then repeating that piece of code for each other element can be tedious to the programmer. Fortunately, a loop allows for writing a single piece of code that can go through every element in an array.

For example, the following program takes five numbers from the user as input and stores them in an array. Afterward, it goes through the elements of the array to calculate the sum of those numbers before printing the sum to the screen.

```
1  using System;
2
3  namespace Add5Numbers
4  {
5      class Program
6      {
7          static void Main()
8          {
9              float[] x = new float[5];
10
11              for(int i = 0; i < 5; i++)
12              {
13                  Console.Write("Value " + (i + 1) + ": ");
14                  x[i] = float.Parse(Console.ReadLine());
15              }
16
17              float sum = 0;
18              for (int i = 0; i < 5; i++) sum += x[i];
19
20              Console.Write("\n" + x[0]);
21
22              for(int i = 1; i < 5; i++)
23              {
24                  Console.Write(" + " + x[i]);
25              }
26
27              Console.WriteLine(" = " + sum);
28          }
29      }
30 }
```

Click to run ►

In the above program, lines 11-15 define a `for` loop that iterates over the array elements. For each element, it asks the user to enter a number at line 13 before storing the entered number into the array element at line 14. Note that the index used in line 14 is `i`, which goes from `0` to `4` as it is incremented by `1` after each iteration.

Line 18 defines a `for` loop that iterates over the elements of the array. For each element, it adds that element to the `sum` variable. After executing this loop, the `sum` variable will be storing the sum of all numbers entered by the user.

Line 20 prints the element at index `0` (i.e. the first element). Lines 22-25 define a `for` loop that iterates over the remaining elements of the array starting by the element at index `1` (i.e. the second element). For each iteration, it prints the `+` character followed by the element value. Finally, line 27 prints the `=` character to the screen followed by the value of `sum`. One output could be:

Output

Value 1: 5

Value 2: 2.8

Value 3: 12

Value 4: 24.5

Value 5: 3.75

5 + 2.8 + 12 + 24.5 + 3.75 = 48.05

7.5 The foreach Loop

Besides, the `while`, `do..while`, and `for` loops (see chapter 6), C# provides the `foreach` loop, which is used exclusively to loop through the elements of an array. The syntax of a `foreach` statement takes the following form:

Syntax

```
foreach (type variable in array) statement
```

To specify a block of statements to be executed in each iteration of the `foreach` loop, the syntax can take the following form:

Syntax

```
foreach (type variable in array)
{
    // The code block to be executed
}
```

A `foreach` loop does not require the programmer to provide a counter to index into the array. In the first iteration, the variable holds the value of the first element in the array (i.e. the value at index 0). In the second iteration, the variable holds the value of the second element in the array (i.e. the value at index 1). In the last iteration, the variable holds the value of the last element in the array.

The following example outputs all elements in the `cars` array:

```
1  using System;
2
3  namespace Cars
4  {
5      class Program
6      {
7          static void Main()
8          {
9              string[] cars = { "Fiat", "Nissan", "BMW" };
10
11              foreach(string car in cars)
12              {
13                  Console.WriteLine(car);
14              }
15          }
16      }
17  }
```

Click to run ►

If you compare the `for` loop and `foreach` loop, you will see that the `foreach` loop is easier to write, it does not require a counter, and it is more readable. However, it has limitations.

A `foreach` loop cannot be used to update array elements. It supports read-only access. If you want to update the elements of an array you can use a `for` or a `while` loop.

7.6 Copying Arrays

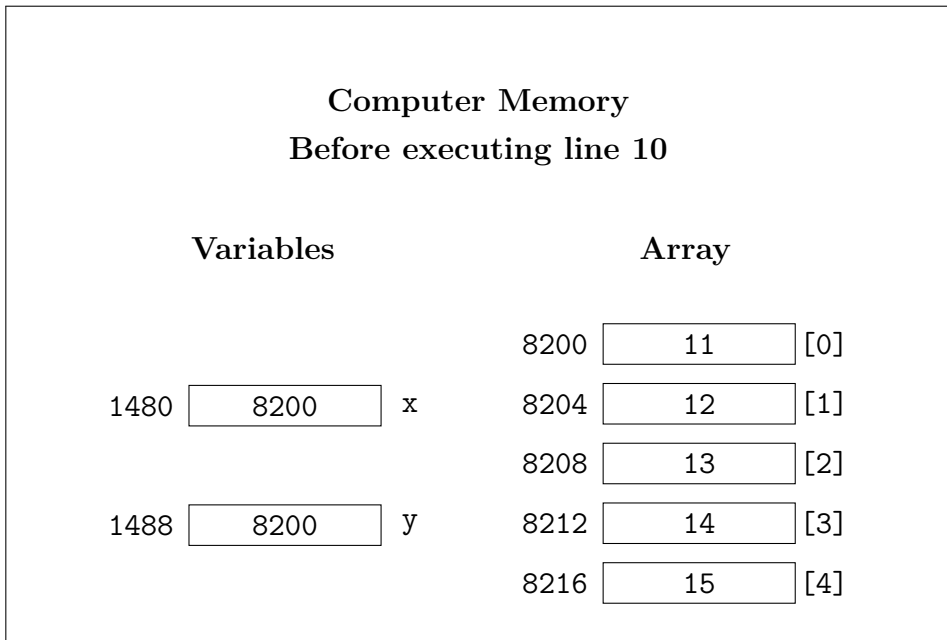
An array variable is a **reference-type** variable. It does not hold the array itself but rather its address in memory. Therefore, copying an array variable means copying the reference it holds not the actual array. For example, consider the following program:

```
1  using System;
2  namespace CopyArrayReference
3  {
4      class Program
5      {
6          static void Main()
7          {
8              int[] x = { 11, 12, 13, 14, 15 };
9              int[] y = x; // Copy the reference
10             y[1] = 70; // Updates the same array
11
12             Console.Write("x:");
13             for (int index = 0; index < 5; index++)
14                 Console.Write("\t" + x[index]);
15
16             Console.WriteLine();
17
18             Console.Write("y:");
19             for (int index = 0; index < 5; index++)
20                 Console.Write("\t" + y[index]);
21         }
22     }
23 }
```

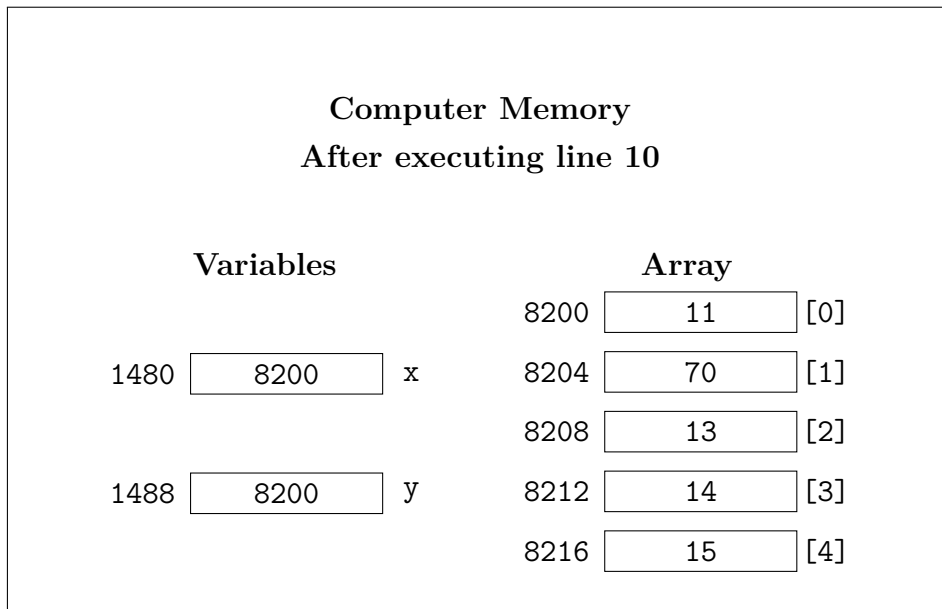
Click to run ►

In the above program, line 8 creates an array of type `int` with five elements (`11`, `12`, `13`, `14` and `15`) and assigns its reference to an array variable named `x`. Line 9 creates an array variable named `y` and assign the contents of `x` to it. Now, `y` holds a copy of the reference stored in `x`. Both `x` and `y` are now holding the address to the same array.

A computer memory consists of a list of bytes and each byte has a unique address. Assuming that the array starts at address `8200` in memory, Both `x` and `y` should store the reference `8200` as illustrated in the following figure. Note that each element in the array takes 4 bytes because this is an `int` array.



Line 10 uses `y` to access the second element of the array (i.e. the element at index `1`) and set its value to `70`. The following figure shows the contents of the computer memory after executing line 10.



Lines 12-14 print the elements of the array using the `index` and the reference stored in `x`. Lines 18-20 print the elements of the array using the `index` and the reference stored in `y`. Both `x` and `y` are referring to the same array. Therefore the output will be:

Output

```
x:  11 70 13 14 15
y:  11 70 13 14 15
```

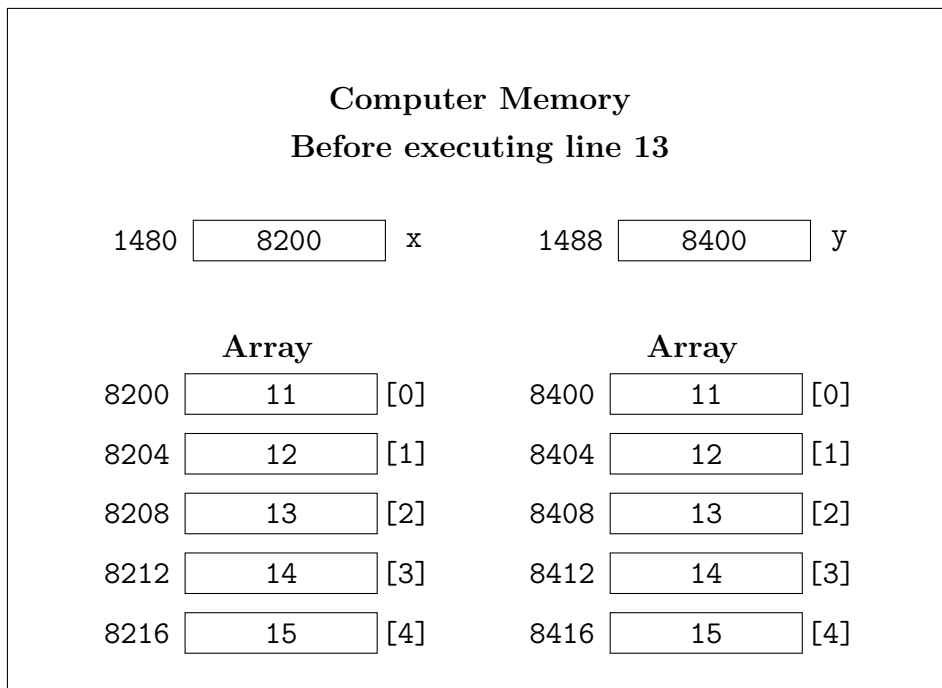
In the previous example, we have copied the array reference not the array itself. To create a copy of an array, we need to create a new array before copying the elements from the old array to the new one. The following example creates a copy of an array.

```
1  using System;
2  namespace CopyArray
3  {
4      class Program
5      {
6          static void Main()
7          {
8              int[] x = { 11, 12, 13, 14, 15 };
9              int[] y = new int[5]; // create a new array
10             for(int index = 0; index < 5; index++)
11                 y[index] = x[index]; // copy elements
12
13             y[1] = 70; // Updates the new array
14
15             Console.Write("x:");
16             for (int index = 0; index < 5; index++)
17                 Console.Write("\t" + x[index]);
18
19             Console.WriteLine();
20
21             Console.Write("y:");
22             for (int index = 0; index < 5; index++)
23                 Console.Write("\t" + y[index]);
24         }
25     }
26 }
```

Click to run ►

In the above program, line 8 creates an array of type `int` with five elements (`11`, `12`, `13`, `14` and `15`) and assigns its reference to an array variable named `x`. Line 9 creates a new array of type `int` with five elements. Lines 10-11 copy the elements of the `x` array into the `y` array.

Assuming that the array defined in line 8 starts at address `8200` in memory, `x` should store the reference `8200` as illustrated in the following figure. Similarly, assuming that the array defined in line 9 starts at address `8400` in memory, `y` should store the reference `8400` as illustrated in the following figure.



Line 13 uses `y` to access the second element (i.e. the element at index `1`) of the array starting at address `8400` and set its value to `70`. The following figure shows the contents of the computer memory after executing line 13. Note that updating the second element of the `y` array did not affect the `x` array because they are two different arrays stored at two different locations in memory.

Computer Memory

After executing line 13

1480

8200

x

1488

8400

y

Array

8200	11	[0]
8204	12	[1]
8208	13	[2]
8212	14	[3]
8216	15	[4]

Array

8400	11	[0]
8404	70	[1]
8408	13	[2]
8412	14	[3]
8416	15	[4]

Creating a copy of an array variable (i.e. a reference to an array) is different from creating a copy of the array itself.

7.7 Multidimensional Arrays

Arrays can have more than one dimension. For example, the following line creates a two-dimensional array of four rows and two columns.

```
1  int[,] x = new int[4, 2];
```

The following line creates an array of three dimensions, 4, 2, and 3:

```
1  int[,,] y = new int[4, 2, 3];
```

The following example creates a two-dimensional array to represent a 4×3 matrix and prints it to the screen.

```
1  using System;
2
3  namespace Matrix
4  {
5      class Program
6      {
7          static void Main()
8          {
9              int[,] x = { { 11, 12, 13 }, { 21, 22, 23 },
10                          { 31, 32, 33 }, { 41, 42, 43 } };
11
```

Click to run ►


```
12         for (int row = 0; row < 4; row++)
13         {
14             for (int col = 0; col < 3; col++)
15             {
16                 Console.Write(x[row, col] + "\t");
17             }
18
19             Console.WriteLine();
20         }
21     }
22 }
23 }
```

In the above program, lines 9-10 create a two-dimensional array of 4 rows and 3 columns before storing the array reference into `x`.

Lines 12-20 use two nested `for` loops to print the elements of the array to the screen. The outer `for` loop (lines 12-20) iterates over the rows and the inner `for` loop (lines 14-17) iterates over the elements of a given row. Line 19 prints a new line after each row. The output of the program should be:

Output

```
11 12 13
21 22 23
31 32 33
41 42 43
```

You can initialize a three-dimensional array and access its elements as

shown in the following example:

```
1 // Three-dimensional array.
2 int[,,,] x = { { { 11, 12, 13 }, { 14, 15, 16 } },
3               { { 17, 18, 19 }, { 20, 21, 22 } } };
4
5 // Accessing array elements.
6 System.Console.WriteLine(x[0, 1, 2]); // Prints 16
7 System.Console.WriteLine(x[1, 0, 1]); // Prints 18
```

The following program allows for adding two matrices. It asks the user to enter the number of rows and columns, creates two arrays, and asks the user to enter the elements of each matrix. Afterward, it prints their sum to the screen.

```
1 using System;
2 namespace AddMatrices
3 {
4     class Program
5     {
6         static void Main()
7         {
8             Console.Write("Enter the number of rows: ");
9             int rows = int.Parse(Console.ReadLine());
10
11             Console.Write("Enter the number of columns: ");
12             int cols = int.Parse(Console.ReadLine());
13
14             Console.WriteLine("\nFirst Matrix");
```

Click to run ►

```
15         double[,] x = new double[rows, cols];
16         for (int i = 0; i < rows; i++)
17             for (int j = 0; j < cols; j++)
18                 {
19                     Console.Write("[{0},{1}] = ", i, j);
20                     x[i,j] =
21                         ↪ double.Parse(Console.ReadLine());
22                 }
23
24         Console.WriteLine("\nSecond Matrix");
25         double[,] y = new double[rows, cols];
26         for (int i = 0; i < rows; i++)
27             for (int j = 0; j < cols; j++)
28                 {
29                     Console.Write("[{0},{1}] = ", i, j);
30                     y[i, j] =
31                         ↪ double.Parse(Console.ReadLine());
32                 }
33
34         Console.WriteLine("\nSum");
35         for (int i = 0; i < rows; i++)
36             {
37                 for (int j = 0; j < cols; j++)
38                     {
39                         double sum = x[i, j] + y[i, j];
40                         Console.Write("{0}\t", sum);
41                     }
42                 Console.WriteLine();
43             }
44     }
```

In the above program, line 15 creates a two-dimensional array named `x`. Lines 16-21 input the elements of `x`. Line 24 creates a two-dimensional array named `y`. Lines 25-30 input the elements of `y`. Lines 33-41 print the sum of the two matrices.

Note that in lines 19, 28, 38, the `Console.Write` method uses the **index component** to print a formatted output to the screen.

A user can use the above program to calculate the sum of the following two matrices:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 \\ 30 & 40 \\ 50 & 60 \end{bmatrix}$$

which should be

$$\begin{bmatrix} 11 & 22 \\ 33 & 44 \\ 55 & 66 \end{bmatrix}$$

In order to do so, the user should enter `3` as the number of rows and `2` as the number of columns before entering the elements of the two matrices. The output of the program should be as follows.

Output

Enter the number of rows: 3

Enter the number of columns: 2

First Matrix

[0,0] = 1

[0,1] = 2

[1,0] = 3

[1,1] = 4

[2,0] = 5

[2,1] = 6

Second Matrix

[0,0] = 10

[0,1] = 20

[1,0] = 30

[1,1] = 40

[2,0] = 50

[2,1] = 60

Sum

11 22

33 44

55 66

Chapter 8

Methods

A **method** is a block of code which only runs when it is called. You can pass data, known as **parameters**, into a method. Methods are used to perform certain actions, and they are also known as **functions**.

Methods (or functions) allow for code reuse. A programmer can define the code once, and use it many times.

In C#, a **class** can have one or more methods. Each method in turn can have one or more statements. A method is executed only if it is called. The `Main` method is the starting point of a C# program. While the `Main` method is executing it might call other methods and those other methods may call other methods, etc.

8.1 Creating a Method

C# provides some predefined methods, which you are already familiar with, such as `System.Console.Write`, but you can also create your own methods to perform certain actions.

For example, the following program creates a method named `PrintHi` besides the `Main` method inside the `Program` class:

```
1  namespace Methods
2  {
3      class Program
4      {
5          static void Main()
6          {
7
8          }
9
10         static void PrintHi()
11         {
12             System.Console.WriteLine("Hi");
13         }
14     }
15 }
```

Click to run ►

In the above code, lines 10-13 define a method named `PrintHi`. Line 10 is called the **head** of the method while lines 11-13 are the **body** of the method.

In line 10, the **method head** consists of the keyword `static` followed by the keyword `void` followed by the **method name** and the **parentheses**. Lines 11-13 constitute the **method body** which is a block of statements to be executed when the method is called.

8.2 Calling a Method

You are already familiar with calling predefined methods, such as `System.Console.Write`. You can also call your own methods to start their execution. For example,

```
1 namespace Methods
2 {
3     class Program
4     {
5         static void Main()
6         {
7             Methods.Program.PrintHi();
8         }
9
10        static void PrintHi()
11        {
12            System.Console.WriteLine("Hi");
13        }
14    }
15 }
```

Click to run ►

In the above program, the execution starts at line 6 (i.e. the first line in body of the `Main` method). Afterward, the execution moves to the next line (i.e. line 7). Line 7 calls the `PrintHi` method. Therefore, the execution jumps from line 7 to line 11 (i.e. the first line in the body of the `PrintHi` method). Afterward, the execution moves to line 12 that prints `Hi` to the screen. Afterward, the execution moves to line 13 (i.e. the last line in the body of the `PrintHi` method). Therefore, the execution jumps back to line 7. Afterward, the execution moves to line 8 (i.e. the last line in the body of the `Main` method). Therefore, the program exits because the `Main` method has completed its execution. The output of the program should be:

Output

"Hi "

In the previous example, the `Main` method calls the `PrintHi` method using the following code line

```
1 Methods.Program.PrintHi();
```

Note that the `Main` method and the `PrintHi` method are both in the same class (i.e. the `Program` class). Therefore, the `Main` method can call the `PrintHi` method simply using

```
1 PrintHi();
```

A method is defined one time but can be called multiple times. The following program calls the `PrintHi` method three times:

```
1 namespace Methods
2 {
3     class Program
4     {
5         static void Main()
6         {
7             PrintHi(); // 1st call
8             PrintHi(); // 2nd call
9             PrintHi(); // 3rd call
10        }
11
12        static void PrintHi()
13        {
14            System.Console.WriteLine("Hi");
15        }
16    }
17 }
```

Click to run ►

In the above program, lines 7-9 call the `PrintHi` method three times in a row. The output should be:

Output

```
Hi
Hi
Hi
```

8.3 Parameters and Arguments

Information can be passed to methods using **parameters**. Parameters act as variables inside the method. They are specified after the method name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma. For example,

```
1 namespace Methods
2 {
3     class Program
4     {
5         static void Main()
6         {
7             PrintHi("Mohamed");
8         }
9
10        static void PrintHi(string name)
11        {
12            System.Console.WriteLine("Hi {0}", name);
13        }
14    }
15 }
```

Click to run ►

In the above program, lines 10-13 define a method named `PrintHi` that has one parameter of type `string` named `name`. Line 7 calls the `PrintHi` method and passes the value `Mohamed` to it as an argument. The output should be:

Output

Hi Mohamed

A **parameter** is a variable in a method definition. It is a placeholder and hence does not have a value. An **argument** is a value passed during method call.

8.4 Multiple Parameters

You can have as many parameters as you like. For example,

```
1 namespace FullName                                     Click to run ►
2 {
3     class Program
4     {
5         static void Main()
6         {
7             PrintName("Mohamed", "Handosa");
8         }
9
10        static void PrintName(string first, string last)
11        {
12            System.Console.Write("{0} {1}", first, last);
13        }
14    }
15 }
```

In the above program, lines 10-13 define a method named `PrintName` that takes two parameters (`first` and `last`). Line 7 calls the `PrintName` method and passes to it two arguments (the first name `Mohamed` and the last name `Handosa`). The output should be:

Output

Mohamed Handosa

The number of arguments in a method call must be the same as the number of parameters in the method definition.

Arguments in a method call must be passed in the same order of the parameters in the method definition.

The type of each argument must be the same as (or implicitly convertible to) the type of its corresponding parameter.

8.5 Return Values

The `void` keyword, used in the examples above, indicates that the method should not return a value. If you want the method to return a value, you can use a data type (e.g. `int` or `double`) instead of `void`, and use the `return` keyword inside the method.

```
1 namespace Add
2 {
3     class Program
4     {
5         static void Main()
6         {
7             int x = 5;
8             int y = 3;
9             int sum = Add(x, y);
10            System.Console.WriteLine(sum);
11        }
12
13        static int Add(int a, int b)
14        {
15            int result = a + b;
16            return result;
17        }
18    }
19 }
```

Click to run ►

In the above code, lines 13-18 define a method named `Add` that takes two parameters (named `a` and `b`) each of type `int` and returns a value of type `int`. Line 9 defines a variable of type `int` named `sum` and assigns to it the result of calling the `Add` method.

The method call in line 9 executes as follows. First the two arguments `x` and `y` are passed to the `Add` method and the execution jumps to line 14. Now, `a` stored a copy of `x` and `b` stored a copy of `y`. Line 15 adds the values of the two variables `a` and `b` and

assigns the result to a variable named `result`. Line 16 uses the `return` keyword to return the value stored in the `result` variable. Afterward, the execution jumps back to line 9. Now the value returned by the `Add` method is assigned to the `sum` variable.

Line 10 prints the value stored in the `sum` variable to the screen. The output should be:

Output

8

A method cannot access the variables defined in another method. The variables defined in a method are created only when that method is called and are deleted from memory once the method returns to its caller.

8.6 Default Arguments

You can use a default argument by assigning a value to a parameter in the method definition using the assignment operator `=`. This makes the parameter **optional**. For example, the following program has a method named `Print` that can print a `string` to the screen either with or without printing a new line after it.


```
1 namespace Printing
2 {
3     class Program
4     {
5         static void Main()
6         {
7             Print("Ziad", true); // Prints new line
8             Print("Malik", false); // No new line
9             Print("Jodi"); // No new line
10        }
11
12        static void Print(string s, bool newLine = false)
13        {
14            if (newLine) Console.WriteLine(s);
15            else Console.Write(s);
16        }
17    }
18 }
```

Click to run ►

In the above program, the method head in line 12 assign a default argument to the parameter named `newLine`. This means that passing an argument to this parameter is now **optional**. Line 7 passes an argument value of `true`. Therefore, `Print` will print a new line after printing `Ziad`. Line 8 passes an argument value of `false`. Therefore, `Print` will not print a new line after printing `Malik`. Line 9 does not pass an argument value for the `newLine` parameter. Therefore, `Print` will use the default argument (i.e. `false`) and will not print a new line after printing `Jodi` to the screen.

8.7 Named Arguments

Passing arguments in correct order can get confusing as the number of parameters increases. In such case, you can use named arguments to make your code more readable. Moreover, with named arguments, the order of the arguments does not matter. For example,

```
1 namespace EmployeeIncome                                Click to run ►
2 {
3     class Program
4     {
5         static void Main()
6         {
7             int x = Income(10000, 7000, 2000);
8
9             int y = Income(
10                 salary: 10000, bonus: 7000, tax: 2000);
11
12             int z = Income(
13                 tax: 2000, salary: 10000, bonus: 7000);
14
15             System.Console.WriteLine("{0}\t{1}\t{2}", x, y, z);
16         }
17
18         static int Income(int salary, int bonus, int tax)
19         {
20             return salary + bonus - tax;
21         }
22     }
23 }
```

In the above program, lines 18-21 define a method named `Income` that takes three parameters named `salary`, `bonus`, and `tax`.

Line 7 calls the `Income` method and passes to it the three arguments `10000`, `7000`, and `2000` in order. It is hard to tell what those numbers stand for by just looking at line 7. A programmer would need to have a look at the method definition to learn about the meaning of those numbers.

Line 8 calls the `Income` method and again passes to it the same three arguments `10000`, `7000`, and `2000` in order. This time, it uses named arguments. This makes the code more readable and it is easy to tell what the arguments stand for by just looking at line 8.

Line 9 calls the `Income` method and passes to it the same three arguments `10000`, `7000`, and `2000`. This time, the arguments are out of order. However, this does not cause a problem because line 9 uses named arguments. The code is still readable and the programmer does not have to pass the arguments in order.

The result will be the same in each of the three method calls. The output of the program should be:

Output		
15000	15000	15000

8.8 Method Overloading

Method overloading allows a class to have multiple methods with the same name if they have different parameters (i.e. different number of parameters or different types of parameters). For example,

```
1  using System;
2
3  namespace MethodOverloading
4  {
5      class Program
6      {
7          static void Main()
8          {
9              Console.WriteLine(Add(5, 3));
10             Console.WriteLine(Add(5, 3, 2));
11             Console.WriteLine(Add(5.4F, 3.2F));
12             Console.WriteLine(Add(155, 3.25F));
13         }
14
15         static int Add(int x, int y) { return x + y; }
16
17         static float Add(float x, float y) {return x + y;}
18
19         static int Add(int x, int y, int z)
20         {
21             return x + y + z;
22         }
23     }
24 }
```

Click to run ►

In the above program, the `Program` class has four methods, which are the `Main` method besides three other methods with the name `Add`. However, those `Add` methods have different parameters.

Line 15 defines a method named `Add` that takes two parameters of type `int`. Line 17 defines a method named `Add` that takes two parameters of type `float`. Lines 19-22 define a method named `Add` that takes three parameters of type `int`.

Line 9 calls the `Add` method passing it two arguments of type `int`. This results in calling the `Add` method defined in line 15. Line 10 calls the `Add` method passing it three arguments of type `int`. This results in calling the `Add` method defined in lines 19-22. Line 11 calls the `Add` method passing it two arguments of type `float`. This results in calling the `Add` method defined in line 17.

Line 12 calls the `Add` method passing it two arguments. The first argument is of type `int` while the second is of type `float`. This results in calling the `Add` method defined in line 17 that can take two `float` arguments because an `int` argument can be implicitly converted to a `float` argument.

C# does not allow two methods, within the same class, to have the same signature. Method **signature** consists of method name, number of parameters, and types of those parameters.

8.9 Passing Arguments

In C#, when passing variables as arguments to a method. Those variables can be passed either by **value** or by **reference**.

Passing a variable to a method by **value** passes a copy of its contents to the method. Thus, the method can change the copy but not the original variable. In all of the previous examples in this chapter, variables were passed to methods by value.

In contrast, passing a variable to a method by **reference** passes the address of the variable in memory. Thus, the method can change the value of the original variable. To pass an argument by reference, use the `ref` keyword, as shown in the following example:

```
1  using System;
2
3  namespace PassByReference
4  {
5      class Program
6      {
7          static void Main()
8          {
9              int x = 5, y = 10;
10
11              Console.WriteLine("Pass by value");
12              Console.WriteLine("Before: x = {0}", x); // 5
13              Increment1(x);
14              Console.WriteLine("After : x = {0}", x); // 5
```

Click to run ►

```
15
16         Console.WriteLine();
17
18         Console.WriteLine("Pass by reference");
19         Console.WriteLine("Before: y = {0}", y); // 10
20         Increment2(ref y);
21         Console.WriteLine("After : y = {0}", y); // 11
22     }
23
24     static void Increment1(int a)
25     {
26         Console.WriteLine("Before: a = {0}", a); // 5
27         a++;
28         Console.WriteLine("After : a = {0}", a); // 6
29     }
30
31     static void Increment2(ref int b)
32     {
33         Console.WriteLine("Before: b = {0}", b); // 10
34         b++;
35         Console.WriteLine("After : b = {0}", b); // 11
36     }
37 }
38 }
```

In the above program, line 13 passes the variable named `x` by value to the `Increment1` method. Now, the parameter named `a` holds a copy of the contents of `x` (i.e. `5`). Line 27 increments `a` but does not affect `x`. Therefore, after executing line 27 `a` becomes `6` but `x` remains `5`.

Line 20 passes the variable named `y` by reference to the method named `Increment2`. Now, the parameter named `b` refers to the same memory location that `y` refers to. This means that the same memory location holding the value `10` has two names now. The `Main` method names it `y` while the `Increment2` method names it `b`. Line 34 increments the contents of the memory location named `b` which is also named `y`. Therefore, after executing line 34 `b` becomes `11` and `y` is of course `11` as well. The output is:

Output

Pass by value

Before: x = 5

Before: a = 5

After : a = 6

After : x = 5

Pass by reference

Before: y = 10

Before: b = 10

After : b = 11

After : y = 11

x	a	y	b
5	6	11	11

8.10 Passing Arrays

An array variable is a reference-type variable that holds the address of the actual array. You can pass an array variable to a method either by value or by reference.

8.10.1 Passing a Reference By Value

Passing a reference-type variable to a method by value creates a copy of that variable. Of course that copy holds the same memory address stored in the original variable. However, changing the copy itself will not affect the original variable. For example,

```
1  using System;
2  namespace ArraysByValue
3  {
4      class Program
5      {
6          static void Main()
7          {
8              int[] x = { 1, 2, 3 };
9
10             Increment(x);
11
12             Console.Write("\nx:\t");
13             foreach (int element in x)
14                 Console.Write("{0}\t", element);
15         }
    
```

Click to run ►

```
16
17     static void Increment(int[] a)
18     {
19         for(int i = 0; i < a.Length; i++) a[i]++;
20
21         a = new int[5];
22
23         Console.Write("a:\t");
24         foreach (int element in a)
25             Console.Write("{0}\t", element);
26     }
27 }
28 }
```

In the above program, line 8 defines a reference-type variable named `x` and creates a 3-elements array before assigning its address to the `x` variable. Now, `x` holds the address of the 3-element array.

Line 10 calls the `Increment` method and passes the `x` variable to it by value. Therefore, the `a` parameter of the `Increment` method becomes a copy of `x` (i.e. `a` refers to the same 3-elements array).

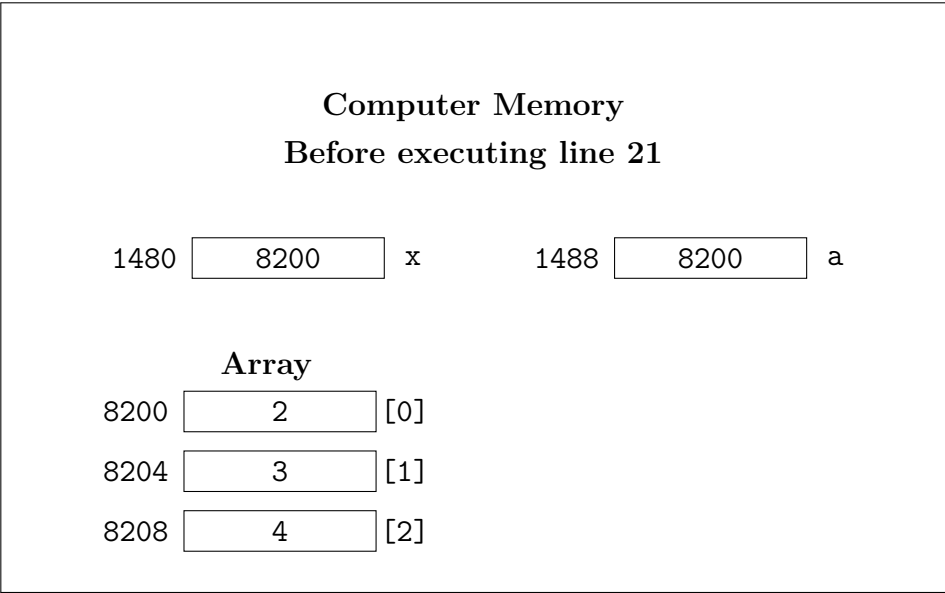
Line 19 uses a `for` loop to iterate over the 3-elements using the reference stored in `a` and increment each element by 1. Note that this changes the same array that `x` refers to.

Line 21 creates a new 5-elements array and assigns its address to `a`. Now, `a` has changed and no longer refers to the 3-element array. Remember that `a` is a copy of `x`. Therefore, changing `a` does

not affect `x`, which is still referring to the 3-elements array. The output of the program should be:

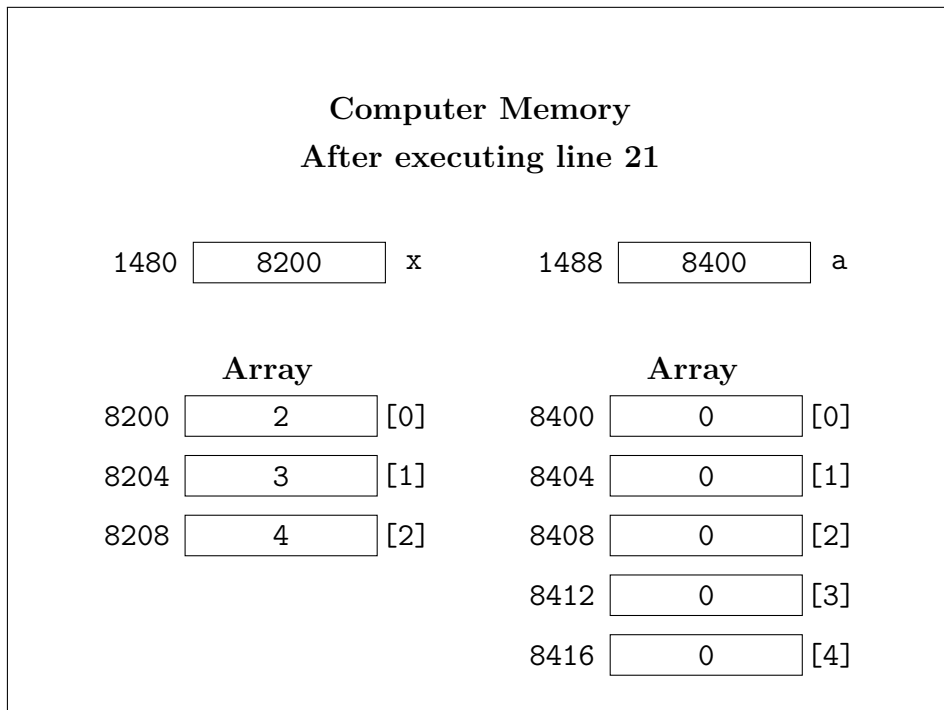
Output					
a:	0	0	0	0	0
x:	2	3	4		

The following figure shows the contents of computer memory before executing line 21 assuming that `x` is at memory location `1480`, `a` is at memory location `1488`, and the memory locations allocated to the 3-elements array start at `8200`. Note that both `x` and `a` are holding the same reference `8200`, which is the address of the first element of the 3-elements array.



The following figure shows the contents of computer memory after executing line 21 assuming that `x` is at memory location `1480`, `a` is at memory location `1488`, the memory locations allocated to the 3-elements array start at `8200`, and the memory locations allocated to the 5-elements array start at `8400`.

Note that `a` holds a different reference now and no longer refers to the 3-elements array. However, `x` is still holding a reference to the 3-elements array.



8.10.2 Passing a Reference By Reference

Passing a reference-type variable to a method by reference sends the original variable itself to the method rather than sending a copy of it. Therefore, any changes that occur to the parameter of the method will affect the original variable. For example,

```
1  using System;
2  namespace ArraysByReference
3  {
4      class Program
5      {
6          static void Main()
7          {
8              int[] x = { 1, 2, 3 };
9              Increment(ref x);
10             Console.Write("\nx:\t");
11             foreach (int element in x)
12                 Console.Write("{0}\t", element);
13         }
14
15         static void Increment(ref int[] a)
16         {
17             for (int i = 0; i < a.Length; i++) a[i]++;
18             a = new int[5];
19             Console.Write("a:\t");
20             foreach (int element in a)
21                 Console.Write("{0}\t", element);
22         }
23     }
24 }
```

Click to run ►

In the above program, line 8 defines a reference-type variable named `x` and creates a 3-elements array before assigning its address to the `x` variable. Now, `x` holds the address of the 3-element array.

Line 9 calls the `Increment` method and passes the `x` variable to it by reference. Therefore, the `a` parameter of the `Increment` method becomes another name to the same location in memory that `x` represents.

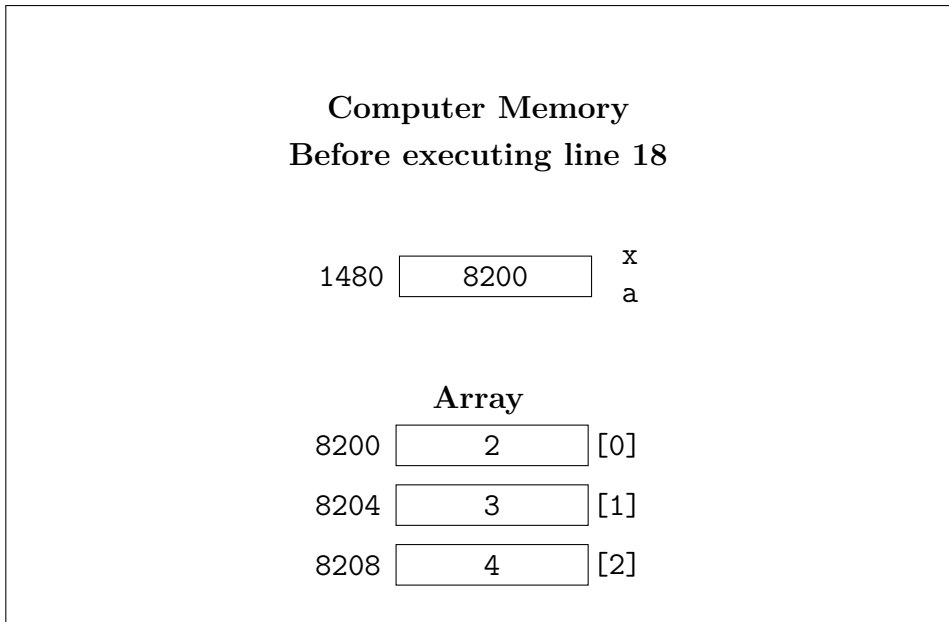
This means that the variable that holds the address of the 3-elements array has two names now, which are `x` according to `Main` and `a` according to `Increment`.

Line 17 uses a `for` loop to iterate over the 3-elements array using the reference stored in `a` and increment each element by 1. Note that this is the same array that `x` refers to.

Line 18 creates a new 5-elements array and assigns its address to `a`. Now, `a` has changed and no longer refers to the 3-element array. Remember that `a` and `x` are two names for the same variable. Therefore, changing `a` changes `x`, which will also refer to the 5-elements array. The output of the program should be:

Output					
a:	0	0	0	0	0
x:	0	0	0	0	0

The following figure shows the contents of computer memory before executing line 18 assuming that `x` is at memory location `1480` and the memory locations allocated to the 3-elements array start at `8200`. Note that `a` represents the same variable that `x` represents. This means that the reference-type variable located at memory address `1480` has two names `x` and `a` and is holding the address of the first element of the 3-elements array, which is `8200`.



The following figure shows the contents of computer memory after executing line 18 assuming that `x` (or `a`) is at memory location `1480`, the memory locations allocated to the 3-elements array start at `8200`, and the memory locations allocated to the 5-elements array

Chapter 9

Case Study

The following example shows a program that collects the scores of a set of students in a number of courses before calculating the average score for each student and printing that average together with the grade to the screen.

```
1  using System;
2
3  namespace AverageScores
4  {
5      class Program
6      {
7          static void Main(string[] args)
8          {
9              // Get the number of students
10             int students = GetStudentsCount();
```

Click to run ►

```
11         // Get the number of courses
12         int courses = GetCoursesCount();
13
14         // Get the scores of all students
15         // in all courses
16         float[,] scores = GetScores(students, courses);
17
18         // Get the average scores for all students
19         float[] averageScores = GetAverageScores(
20             students, courses, scores);
21
22         // Print the average score and grade
23         // for each students
24         PrintGrades(students, averageScores);
25     }
26
27
28     static int GetStudentsCount()
29     {
30         int students;
31
32         do
33         {
34             // Ask about the number of students
35             Print("How many students? ");
36
37             // input the number of students
38             students = GetInt();
39
40             } while (students <= 0); // Repeat until valid
41
42         return students;
43     }
44
```

```
45     static int GetCoursesCount()
46     {
47         int courses;
48         do
49         {
50             // Ask about the number of courses
51             Print("How many courses? ");
52
53             courses = GetInt(); // number of courses
54         } while (courses <= 0); // Repeat until valid
55         return courses;
56     }
57
58     static float[,] GetScores(
59         int students, int courses)
60     {
61         // Create a two-dimensional array
62         float[,] scores = new float[students, courses];
63
64         // Iterate over students
65         for (int i = 0; i < students; i++)
66         {
67             PrintLine("Scores of student " + (i + 1));
68
69             // Iterate over the courses of student i
70             for (int j = 0; j < courses; j++)
71             {
72                 // Ask about course score
73                 Print("Student " + (i + 1)
74                     + " --> course " + (j + 1) + ": ");
75
76                 // Input course score
77                 scores[i, j] = GetFloat();
78             }
```

```
79
80         // Print a new line before the next student
81         Console.WriteLine();
82     }
83
84     return scores;
85 }
86
87 static float[] GetAverageScores(
88     int students, int courses, float[,] scores)
89 {
90     // Create a one-dimensional array
91     float[] averageScores = new float[students];
92
93     // Iterate over students
94     for (int i = 0; i < students; i++)
95     {
96         averageScores[i] = 0;
97
98         // Loop over courses
99         for (int j = 0; j < courses; j++)
100         {
101             // Add course score
102             averageScores[i] += scores[i, j];
103         }
104
105         // Divide the total by the number
106         // of courses to get the average
107         averageScores[i] /= courses;
108     }
109
110     return averageScores;
111 }
112
```

```
113     static void PrintGrades(  
114         int students, float[] averageScores)  
115     {  
116         // Iterate over students to print grade  
117         for (int i = 0; i < students; i++)  
118         {  
119             // Print average score  
120             Print("Student " + (i + 1)  
121                 + ": "  
122                 + averageScores[i] + " ");  
123  
124             // Print grade  
125             if (averageScores[i] >= 85)  
126             {  
127                 PrintLine("Excellent");  
128             }  
129             else if (averageScores[i] >= 75)  
130             {  
131                 PrintLine("Very Good");  
132             }  
133             else if (averageScores[i] >= 65)  
134             {  
135                 PrintLine("Good");  
136             }  
137             else if (averageScores[i] >= 50)  
138             {  
139                 PrintLine("Pass");  
140             }  
141             else  
142             {  
143                 PrintLine("Fail");  
144             }  
145         }  
146     }
```

```
147
148     static void Print(string s)
149     {
150         Console.Write(s);
151     }
152
153     static void PrintLine(string s)
154     {
155         Console.WriteLine(s);
156     }
157
158     static int GetInt()
159     {
160         string s = Console.ReadLine();
161         int value = int.Parse(s);
162         return value;
163     }
164
165     static float GetFloat()
166     {
167         return float.Parse(Console.ReadLine());
168     }
169 }
170 }
```

